

UNIVERSITY OF SOUTHERN DENMARK

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

SPDM801: MASTER THESIS IN COMPUTER SCIENCE

Formalising π LL in Lean

Author

Mikkel Brix Nielsen
mikke21

Supervisors

Supervisor: Fabrizio Montesi
Cosupervisor: Marco Peressotti
Cosupervisor: Xueying Qin

1st June 2026



Abstract

This thesis presents a mechanised formalisation of π LL, a process calculus relating linear logic and session-typed communication, in Lean 4. The formalisation covers the static structure of the full calculus, including session types, processes, environments, hyperenvironments, typing judgements, and substitution principles. To represent binding, the development retains explicit free identifiers while encoding bound occurrences by indices: channel binders are handled through a locally nameless representation with co-finite premises, while polymorphic type binders are treated through type-depth indexing, shifting, and substitution. The operational semantics and session-fidelity development are mechanised for the multiplicative fragment, π MLL, using a derivation-level transition system together with projected process-level and environment-level relations. The completed metatheory establishes the projection of typed transitions to process and resource transitions and develops the reverse lifting argument for all multiplicative cases except the restriction case. The remaining *res* case is isolated to the reconstruction of endpoint-bearing hyperenvironment components after a transition under restriction. The development demonstrates that mechanising the paper theory in this way can relocate implicit reasoning about bound names, freshness, and unordered linear resources into explicit structural invariants. It thereby contributes a mechanised account of π LL and identifies the remaining infrastructure required to complete session fidelity for π MLL.

Resumé

Denne afhandling præsenterer en mekaniseret formalisering af π LL, en proceskalkule, der forbinder lineær logik med sessionstypet kommunikation, i Lean 4. Formaliseringen dækker den statiske struktur af den fulde kalkule, herunder sessionstyper, processer, miljøer, hypermiljøer, *typing judgements* og substitutionsprincipper. Til repræsentation af binding fastholder udviklingen eksplicitte frie navne, mens bundne forekomster repræsenteres ved indeks: kanalbindere håndteres gennem en *locally nameless* repræsentation med co-finite-præmisser, mens polymorfe typebindere behandles gennem typedybdeindeksering, forskydning og substitution. Den operationelle semantik og udviklingen af *session fidelity* er mekaniseret for det multiplikative fragment, π MLL, ved hjælp af et transitionssystem over *typing derivations* samt projicerede transitionsrelationer for processer og hypermiljøer. Den gennemførte metateori etablerer projektionen af typede transitioner inkluderende til proces- og ressourcetransitioner og udvikler det omvendte løfteargument for alle multiplikative tilfælde undtagen restriktionstilfældet. Det resterende *res*-tilfælde er isoleret til rekonstruktionen af de hypermiljøkomponenter, der bærer de restriktede endepunkter, efter en transition under restriktion. Udviklingen viser, at en mekanisering af papirteorien på denne måde kan flytte implicit ræsonnement om bundne navne, friskhed og uordnede lineære ressourcer til eksplicitte strukturelle invarianter. Afhandlingen bidrager dermed med en mekaniseret redegørelse for π LL og identificerer den resterende infrastruktur, der kræves for at fuldføre sessionsfidelitet for π MLL.

CONTENTS

1	Introduction	3
2	Background	4
2.1	Classical linear logic, CLL	4
2.2	The π -calculus: processes and communication	5
2.3	Types and propositions-as-sessions correspondence	6
2.4	Environments, hyperenvironments and judgements	7
2.5	Operational semantics	8
2.6	Process semantics and erasure	11
2.7	Semantics of typing environments	12
2.8	Managing bound variables	13
3	Binding architecture	14
3.1	Design goals and the approach	14
3.2	The locally nameless representation	15
3.3	Opening and closing binders	16
3.4	Co-finite quantification	17
4	Formalisation	19
4.1	Model	19
4.2	Semantics	32
5	Metatheory	35
5.1	Session fidelity	35
6	Discussion	38
6.1	Proof architecture	38
6.2	Proof-engineering trade-offs	38
6.3	Structural congruence	39
6.4	Current limitations	40
7	Future work	40
7.1	Completing session fidelity for π MLL	40
7.2	Extending to full π LL	41
7.3	Structural congruence	42
7.4	Readiness, productivity and behavioural metatheory	42
8	Conclusion	43
	References	44
A	Code availability	45
B	Lean file structure	45
C	Common Lean notation	45
D	Selected core definitions	49
D.1	Session types	49
D.2	Processes	49
D.3	Judgements	50
D.4	Transition labels	52
D.5	Transition systems	53
E	Session fidelity theorems	57
F	Selected proof infrastructure	58

1 Introduction

The Curry-Howard correspondence establishes a deep connection between logic and computation, where proofs correspond to programs and logical propositions correspond to types. This perspective has been particularly influential in the study of concurrent systems, where linear logic and session types provide a logical foundation for structured communication.

In this setting, linear logic captures resource-sensitive reasoning, while session types describe communication protocols between concurrent processes. The resulting correspondence, often referred to as propositions-as-sessions, provides a principled interpretation of interaction as logical proof transformation, and has been further related to process calculi such as the π -calculus.

This line of work has been further developed by [Montesi and Peressotti \[2021\]](#), who provide a unified account reconciling linear logic, the π -calculus, and their metatheory. Their framework serves as the formal basis for the system π LL considered in this thesis.

Motivation and correctness concerns. Although these theoretical correspondences provide a precise account of the relationship between proofs, types, and processes, their paper-based presentations are prone to subtle errors. In particular, applications of inference rules in linear logic and session-typed systems may rely on ordering conventions, freshness assumptions, or intuitive readings of syntax that are not checked mechanically. As a result, small mistakes in how a rule is used can remain difficult to detect in written derivations.

As an illustrative example, a small mistake in the application of a typing rule can arise when the intuitive order of names does not match the formal order imposed by the rule. It is tempting to assign types following the order in which names appear syntactically, but a rule may assign types according to the role each name plays instead. Thus, despite the rule itself being correct, its application is easy to get wrong because the intuitive and formal interpretations pull in opposite directions. Such mistakes do not reflect a flaw in the underlying theory, but they are easy to overlook in paper-based presentations where rule application is guided by intuition rather than by strict syntactic structure. This makes the system a natural candidate for mechanisation, since in proof assistants such as Lean, every rule application must instantiate the required variables and side conditions explicitly.

These observations motivate a more disciplined approach to formal reasoning, where correctness is ensured by construction rather than by inspection.

Why mechanisation. Mechanisation provides a means of bridging this gap. Rather than extending the underlying theory, the goal of mechanisation in this work is to validate and internalise existing results within a proof assistant. This allows us to eliminate ambiguity arising from paper-based presentation and to ensure that all derivations are checked with respect to the system's formal semantics.

Contributions. In this thesis, we present a mechanised formalisation of the π -calculus-based linear logic system π LL as developed by Montesi and Peressotti, with particular emphasis on its multiplicative fragment. The contributions are as follows:

- A formalisation of the syntax and typing system of π LL in Lean, including processes, session types, environments, hyperenvironments, typing judgements, and substitution operations for both names and types.
- A binding architecture that represents bound occurrences by indices while retaining explicit free identifiers, with co-finite opening for channel binders and depth-indexed treatment of polymorphic type binders.
- A formalisation of the operational semantics for the multiplicative fragment of π LL.

- A mechanised development of the session-fidelity proof architecture for the multiplicative fragment, with the remaining proof obligation isolated to the *res* case.

Scope of the formalisation. This thesis focuses on a mechanised reconstruction of the π LL system of [Montesi and Peressotti \[2021\]](#), with particular emphasis on its multiplicative fragment. The syntactic and typing components of π LL have been mechanised for the full system, including processes, session types, typing judgements, and substitution operations. The operational semantics are defined for the multiplicative fragment, for which the session-fidelity development addresses the project objective of relating process evolution to the linear-logic protocols represented by typing derivations. Within this fragment, completing the session-fidelity argument is reduced to the restriction case, *res*, in which a transition occurs under a restricted session.

Consequently, metatheoretic results concerning the static typing infrastructure apply to the full system, while the operational and session-fidelity results developed in this thesis are restricted to the multiplicative fragment and remain conditional on completion of the outstanding *res* case. The aim of this work is not to extend the theory, but to provide a mechanised account of an existing theoretical framework, to establish the completed parts of its metatheory in Lean, and to isolate the remaining proof obligation for the targeted session-fidelity result.

2 Background

This section begins by introducing the theoretical foundations on which the formalisation is based, then following [Montesi and Peressotti \[2021\]](#) in the present of the full π LL calculus, including processes, types, typing, environments / hyperenvironments, and judgements, then restrict attention to the multiplicative fragment π MLL when discussing operational semantics.

Following [Montesi and Peressotti \[2021\]](#), we use *blue* to highlight types and *red* for process terms throughout this thesis.

2.1 Classical linear logic, CLL

Linear logic [[Girard 1987](#)] is a refinement of classical and intuitionistic logic. In contrast to these systems, where assumptions can be duplicated or discarded freely, linear logic controls the use of structural rules such as weakening and contraction, thereby giving formulas a resource-sensitive interpretation [[Di Cosmo and Miller 2023](#)]. In this thesis, we work in the classical presentation of linear logic, where sequents are symmetric and duality replaces the distinction between assumptions and conclusions.

The multiplicative fragment forms the core of the system and includes the tensor connective $A \otimes B$ and its dual $A \wp B$, together with the corresponding units 1 and $\perp$. These connectives capture composition and interaction of resources.

Intuitively, $A \otimes B$ describes the composition of two independent resources, while $A \wp B$ captures their dual form of interaction. Under a process interpretation, $A \otimes B$ can be read as producing a value of type A and continuing as B , whereas $A \wp B$ corresponds to receiving a value of type A and proceeding as B . The units 1 and $\perp$ represent the absence of further output and input, respectively. A simplified presentation of these rules is given below:

$$\frac{}{\vdash 1} \quad \frac{}{\vdash \Gamma, \perp} \quad \frac{\vdash \Gamma, A \quad \vdash \Delta, B}{\vdash \Gamma, \Delta, A \otimes B} \otimes \quad \frac{\vdash \Gamma, A, B}{\vdash \Gamma, A \wp B} \wp$$

Here *rule* $\otimes$ presents a simplified form of the tensor rule from classical linear logic. In general, the tensor connective $A \otimes B$ combines two independent derivations. It requires a proof of A and a proof of B , each obtained from disjoint contexts, and produces a proof of the composite resource

$A \otimes B$. Intuitively, this corresponds to combining two independent resources into a single composite resource, and can be read as producing a value of type A and then continuing as B .

The unit rule [rule 1](#) introduces the unit 1 , which represents the absence of further obligations. It can be understood as an *empty output*, corresponding to a computation that performs no further interaction, and serves as the neutral element of tensor.

Dually, [rules \$\perp\$](#) and [\$\wp\$](#) describe the behavior of the connective $A \wp B$ and its unit $\perp$. The [\$\wp\$](#) rule combines resources within a single context, reflecting a form of interaction where both components are made available to the surrounding context. From an operational perspective, this corresponds to interacting with an input of type A while continuing as B . The unit $\perp$ represents the absence of further input, acting as an *empty input* and serving as the dual neutral element.

For example, two independent instances of the unit 1 can be introduced by the [1](#) rule and combined via the [\$\otimes\$](#) rule to derive $1 \otimes 1$. In this case the surrounding context Γ and Δ are empty:

$$\frac{\frac{}{\vdash 1} 1 \quad \frac{}{\vdash 1} 1}{\vdash 1 \otimes 1} \otimes$$

Here each leaf introduces one resource independently, and the tensor rule combines them into a single compound resource $1 \otimes 1$. After this step, the individual resources are no longer available separately, reflecting the resource-sensitive nature of the system.

This resource-sensitive view of logic provides a foundation for a use in computer science, particularly in the study of concurrency. In the following sections, this perspective is used to motivate the process interpretation underlying π LL.

2.2 The π -calculus: processes and communication

The π -calculus [[Milner et al. 1992](#)] is a process calculus for modeling concurrent computation with dynamic communication topology, where channel names themselves can be transmitted, allowing the communication network to evolve during execution. We follow the presentation of [Montesi and Peressotti \[2021\]](#), which adopts [Wadler](#)'s convention of using square brackets '['-']' for output and round parentheses '(-)' for input.

Programs in π LL are processes $(P, Q, R, \dots)$, which communicate over names $(x, y, z, \dots)$ representing session endpoints. Sessions are binary, a discipline enforced by the typing system introduced in [Section 2.3](#). Following [Montesi and Peressotti \[2021\]](#), we assume an infinite set of names with decidable equality. The process terms of π LL are defined by the following grammar, reproduced from [Montesi and Peressotti \[2021\]](#) with minor adjustments:

$P, Q ::=$	$x[y].P$	output y on x and continue as P
	$x(y).P$	input y on x and continue as P
	$x[].P$	output (empty message) on x and continue as P
	$x().P$	input (empty message) on x and continue as P
	$\nu xy.P$	name restriction (cut)
	$P \mid Q$	parallel composition
	$x[L].P$	select left on x and continue as P
	$x[R].P$	select right on x and continue as P
	$x.\text{case}\{L:P, R:Q\}$	offer a binary choice between P or Q on x
	$x[A].P$	output type A on x and continue as P
	$x(X).P$	input a type as X on x and continue as P
	$!x\langle z_1, \dots, z_n \rangle.P$	server (replicable process) on x depending on servers on $z_1 \dots z_n$

	$x[\text{USE}].P$	consume a server on x and continue as P
	$x[\text{DUP}](x').P$	duplicate server x as x' in P
	$x[\text{DISP}].P$	dispose of a server on x
	$x \leftrightarrow y$	link x and y
	0	terminated process

Term $x[y].P$ allocates a fresh name y , outputs it over x , and proceeds as P . Dually, $x(y).P$ takes input over x , binding it to y in P . Terms $x[].P$ and $x().P$ model empty output and input. Restriction $\nu xy.P$ connects the two endpoints x and y of P to form a session, where the order of x and y is irrelevant and $\nu xy.P \equiv \nu yx.P$. Parallel composition $P \mid Q$ runs P and Q concurrently, and 0 is the terminated process, acting as the unit of parallel composition.

Example 2.1. Consider the process $P \triangleq \nu xz(x[].0 \mid z().y[].0)$. This process creates a private session between the restricted endpoints x and z . The left process, $x[].0$, signals termination on x before terminating, while the right process, $z().y[].0$, waits for a termination signal on z before proceeding by sending a termination signal on y and then terminating.

This calculus provides the operational foundation for πLL . In the following section, we introduce the type system that governs how names are used, ensuring that each session endpoint is used exactly according to its protocol.

2.3 Types and propositions-as-sessions correspondence

In πLL , session types $(A, B, C, \dots)$ are linear logic propositions [Caires and Pfenning 2010; Montesi and Peressotti 2021; Wadler 2014]. Each type describes the communication protocol of a channel endpoint, and duality ensures that the two endpoints of every session implement complementary protocols.

The account of session types used in this thesis follows the presentation of Montesi and Peressotti [2021], which builds on the propositions-as-sessions correspondence developed by Carbone et al. [2016] and Wadler [2014]. In this correspondence, each connective of linear logic is interpreted as a communication protocol, where $A \otimes B$ describes sending a value of type A and continuing as B , while its dual $A \wp B$ describes receiving. The units 1 and $\perp$, again, represent the absence of out and input, respectively. The additive connectives $A \oplus B$ and $A \& B$ capture choice in the sense of selection and offering of alternatives, while the exponentials $!A$ and $?A$ govern replication. The $\forall X.A$ and $\exists X.A$ allow polymorphic session behavior. The type grammar of πLL is defined by Montesi and Peressotti [2021] as follows:

$A, B ::=$	$A \otimes B$	send A , proceed as B		$A \wp B$	receive A , proceed as B
	1	empty output, unit for $\otimes$		$\perp$	empty input, unit for $\wp$
	$A \& B$	offer A or B		$A \oplus B$	select A or B
	$\exists X.A$	existential, type output		$\forall X.A$	universal, type input
	$?A$	client request		$!A$	server accept
	X	type variable		$X^\perp$	dual of type variable

Duality is defined structurally on session types by exchanging each connective with its dual and forms an involution, specifically $(A^\perp)^\perp = A$. Consequently, the endpoints of a well-typed session are always assigned dual types, ensuring they are compatibility:

$$(A \otimes B)^\perp = A^\perp \wp B^\perp \quad 1^\perp = \perp \quad (A \oplus B)^\perp = A^\perp \& B^\perp$$

$$(!A)^\perp = ?A^\perp \quad (\exists X.A)^\perp = \forall X.A^\perp \quad (X)^\perp = X^\perp$$

2.4 Environments, hyperenvironments and judgements

Typing environments in π LL are organised into two layers. Environments track how individual names are typed, while hyperenvironments track which names are used independently in parallel.

Following the presentation in Montesi and Peressotti [2021], *environments* $(\Gamma, \Delta, \Xi, \dots)$ are defined as lists allowing the exchange, meaning order is irrelevant. They can be written as $\Gamma = x_1 : A_1, \dots, x_n : A_n$, where each x_i is associated to its respective A_i , for $1 \leq i \leq n$. Environments can be merged as long as they do not share any names for example assume Γ and Δ do not share any names, then Γ, Δ is the environment containing exactly the entries of Γ and Δ regardless of ordering. Environments act as a partial commutative monoid, using “,” as sum and the empty environment $\bullet$ as unit:

$$\Gamma, \bullet = \Gamma \quad \Gamma, \Delta = \Delta, \Gamma \quad (\Gamma, \Delta), \Xi = \Gamma, (\Delta, \Xi)$$

Environments dictate how names are used, but it does not distinguish whether names are used independently or in parallel. That role is handled by *hyperenvironments* $(\mathcal{G}, \mathcal{H}, \mathcal{I}, \dots)$, which are collections of environments joined using the parallel operator ‘|’. They can be written as follows $\mathcal{G} = \Gamma_1, \dots, \Gamma_n$, where $\Gamma_1, \dots, \Gamma_n$ is a collection of environments. Given $\mathcal{G}$ and $\mathcal{H}$ having no names in common then $\mathcal{G} | \mathcal{H}$ is the hyperenvironment containing exactly the environments of $\mathcal{G}$ and $\mathcal{H}$, ignoring order. Like environments, all names in a hyperenvironment are unique, they can be combined as long as they do not share any names, and they act as a partial commutative monoid using ‘|’ as sum and $\emptyset$ as unit, where $\emptyset$ is the hyperenvironment containing no environments:

$$\mathcal{G} | \emptyset = \mathcal{G} \quad \mathcal{G} | \mathcal{H} = \mathcal{H} | \mathcal{G} \quad (\mathcal{G} | \mathcal{H}) | \mathcal{I} = \mathcal{G} | (\mathcal{H} | \mathcal{I})$$

Judgements, written as $\vdash P :: \mathcal{G}$, states that the process P uses its free names in accordance with $\mathcal{G}$ and can be derived using the inference rules displayed in Figure 1.

Typing Derivations $(\mathcal{D}, \mathcal{E}, \mathcal{F}, \dots)$ are written as $\vdash P :: \mathcal{G}$, indicating that $\mathcal{D}$ is a derivation for the judgement.

The multiplicative rules form the core of the system. **Rule cut** composes two processes in P sharing dual endpoints $x : A$ and $y : A^\perp$, hiding them via restriction. This is the logical equivalent of communication. **Rule mix** and **rule mix₀** compose independent processes in parallel and introduce the terminated process, respectively.

Rules $\otimes$ and **$\wp$** type output and input. Note that **rule $\otimes$** types the sent name y in a *separate* environment from the continuation, reflecting that the two are parallel resources. By contrast, **rule $\wp$** keeps y and the continuation x in the *same* environment, reflecting their sequential dependency. The unit **rules 1** and **$\perp$** type empty output and empty input, where **rule 1** requires the continuation to be well-typed in the empty hyperenvironment, meaning P is necessarily semantically equivalent to **0**.

The additive rules (**$\oplus_1$** , **$\oplus_2$** and **$\&$**) type selection and offering of alternative processing paths. Note that **rule $\&$** requires both branches to be typed in the *same* context Γ , reflecting that the choice of branch is made by the other endpoint.

The exponential rules type server replication. The **rule !** requires all free names except x to be typed using $?$, enforcing that a server only depends on other servers. The **rules $?$** , **w** and **c** allow a client to use, discard, or duplicate a server respectively.

The **rule ax** types a link between two dual endpoints, $x \leftrightarrow y$, forwarding all messages sent on x safely to y and vice versa. **Rule $\exists$** and **rule $\forall$** type polymorphic communication, where the side condition $X \notin \text{ft}(\Gamma)$ in **rule $\forall$** , ensures the introduced type variable, X , does not accidentally shadow one already existing in Γ .

Multiplicative Fragment

$$\begin{array}{c}
\frac{\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^\perp}{\vdash \mathbf{v}xyP :: \mathcal{G} \mid \Gamma, \Delta} \text{CUT} \quad \frac{\vdash P :: \mathcal{G} \quad \vdash Q :: \mathcal{H}}{\vdash P \mid Q :: \mathcal{G} \mid \mathcal{H}} \text{MIX} \quad \frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \\
\frac{\vdash P :: \Gamma, y : A \mid \Delta, x : B}{\vdash x[y].P :: \Gamma, \Delta, x : A \otimes B} \otimes \quad \frac{\vdash P :: \emptyset}{\vdash x[] . P :: x : 1} 1 \quad \frac{\vdash P :: \Gamma, y : A, x : B}{\vdash x(y).P :: \Gamma, x : A \wp B} \wp \quad \frac{\vdash P :: \Gamma}{\vdash x().P :: \Gamma, x : \perp} \perp
\end{array}$$

Additional rules for Additives, Quantifiers and Exponentials

$$\begin{array}{c}
\frac{\vdash P :: \Gamma, x : A}{\vdash x[L].P :: \Gamma, x : A \oplus B} \oplus_1 \quad \frac{\vdash P :: \Gamma, x : B}{\vdash x[R].P :: \Gamma, x : A \oplus B} \oplus_2 \quad \frac{\vdash P :: \Gamma, x : A \quad \vdash Q :: \Gamma, x : B}{\vdash x.\text{case}\{L:P, R:Q\} :: \Gamma, x : A \& B} \& \\
\frac{}{\vdash x \leftrightarrow y :: x : A^\perp, y : A} \text{AX} \quad \frac{\vdash P :: \Gamma, x : A}{\vdash x[\text{USE}].P :: \Gamma, x : ?A} ? \quad \frac{\vdash P :: \Gamma}{\vdash x[\text{DISP}].P :: \Gamma, x : ?A} \text{W} \quad \frac{\vdash P :: \Gamma, x : ?A, x' : ?A}{\vdash x[\text{DUP}](x').P :: \Gamma, x : ?A} \text{C} \\
\frac{\vdash P :: z_1 : ?B_1, \dots, z_n : ?B_n, x : A}{\vdash !x\langle z_1, \dots, z_n \rangle . \{P\} :: z_1 : ?B_1, \dots, z_n : ?B_n, x : !A} ! \quad \frac{\vdash P :: \Gamma, x : B\{A/X\}}{\vdash x[A].P :: \Gamma, x : \exists X.B} \exists \quad \frac{\vdash P :: \Gamma, x : B \quad X \notin \text{ft}(\Gamma)}{\vdash x(X).P :: \Gamma, x : \forall X.B} \forall
\end{array}$$

Fig. 1. π LL, typing rules, adapted from Figure 2 and Figure 6 of Montesi and Peressotti [2021].

Example 2.2. The process from Example 2.1 has the typing $\vdash \mathbf{v}zx(z()).y[].\mathbf{0} \mid x[].\mathbf{0} :: y : 1$, as shown by the derivation below:

$$\begin{array}{c}
\frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash \mathbf{0} :: \emptyset} \text{MIX}_0 \quad \frac{}{\vdash y[].\mathbf{0} :: y : 1} 1 \\
\frac{\vdash \mathbf{0} :: \emptyset}{\vdash x[].\mathbf{0} :: x : 1} 1 \quad \frac{\vdash y[].\mathbf{0} :: y : 1}{\vdash z().y[].\mathbf{0} :: y : 1, z : \perp} \perp \\
\frac{\vdash x[].\mathbf{0} \mid z().y[].\mathbf{0} :: x : 1 \mid y : 1, z : \perp}{\vdash \mathbf{v}zx(x[].\mathbf{0} \mid z().y[].\mathbf{0}) :: y : 1} \text{MIX} \quad \text{CUT}
\end{array}$$

Note that, for the typing environment $x : 1 \mid y : 1, z : \perp$ to align with the premise of rule CUT, we utilise the monoidal properties of environments and hyperenvironments. In particular, using their identity laws, we instantiate the meta-variables of the rule as $\mathcal{G} = \emptyset$ and $\Gamma = \bullet$. Under these instantiations, the premise of rule CUT: $\vdash P :: \mathcal{G} \mid \Gamma, x : A \mid \Delta, y : A^\perp$ can be viewed as $x : A \mid \Delta, y : A^\perp$. This effectively instantiates the CUT over the dual types $A = 1$ and $A^\perp = \perp$ on channels x and z , with $\Gamma = y : 1$, while ensuring the restricted endpoints implement dual session behaviours.

2.5 Operational semantics

The operational semantics of π LL are given as a labelled transition system (LTS) defined over *typing derivations*, in which processes evolve by performing actions on their free names. The semantics follow a Structural Operational Semantics (SOS) style presentation and form the basis for the metatheoretic results of the calculus. We will be focusing on the multiplicative fragment for this part, which is given in Figure 2. The full LTS is presented in Figure 2 and Appendix B.2 of Montesi and Peressotti [2021].

The operational semantics rely explicitly on the structural equivalence (rule $=_\alpha$) to rename restricted variables via α -equivalence prior to a transition. In paper-based presentations, α -equivalence is often handled implicitly through Barendregt's Variable Convention. In a mechanised setting,

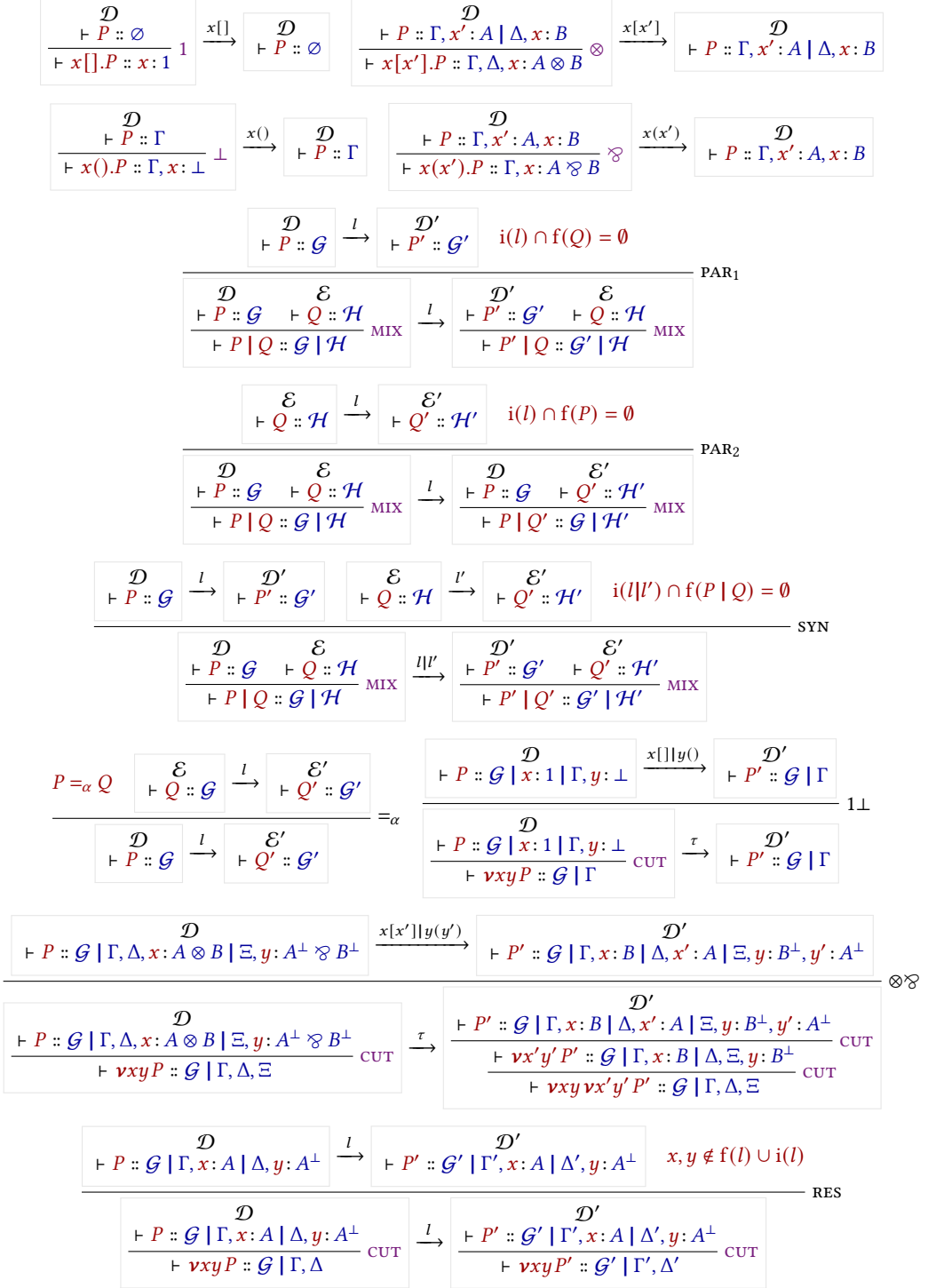


Fig. 2. π MLL, transition rules for derivations, as presented in Figure 2 of Montesi and Peressotti [2021].

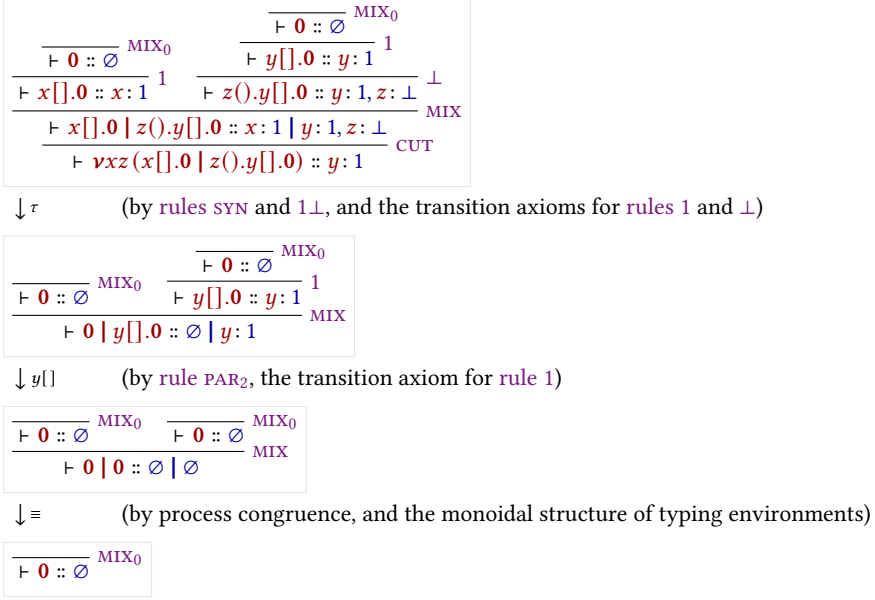


Fig. 3. An execution for the derivation in Example 2.2.

however, reasoning over named variables requires freshness conditions, capture-avoiding substitution, and explicit renaming lemmas, introducing proof overhead [Aydemir et al. 2005; Charguéraud 2012]. As discussed in Section 3, this motivates the use of a different binding representation for the syntax of π LL, one designed to reduce explicit α -equivalence reasoning and simplify the treatment of bound variables.

Actions are defined in the set $Act = \{x[], x(), x[y], x(y) \mid x, y \text{ names}\}$, representing empty output, empty input, name output, and name input respectively. These come together with the silent label τ for internal communication and the parallel composition of labels $l \mid l'$ to form the set of transition labels, defined as $Lbl = \{\tau, l, (l \mid l') \mid l, l' \in Act, i(l) \cap i(l') = \emptyset\}$. The side condition imposed on the parallel composition of labels exists to ensure two labels do not introduce the same name, which would break linear resource management.

The free and introduced names of labels are defined by $f(x[]) = f(x()) = f(x[y]) = f(x(y)) = \{x\}$, $f(\tau) = i(\tau) = \emptyset$, and $f(l \mid l') = f(l) \cup f(l')$, and introduced names by $i(x[]) = i(x()) = \emptyset$, $i(x[y]) = i(x(y)) = \{y\}$, and $i(l \mid l') = i(l) \cup i(l')$. The parallel label $l \mid l'$ additionally requires $i(l) \cap i(l') = \emptyset$, ensuring no two components introduce the same name.

Example 2.3. Figure 3 shows a possible execution for the derivation of the process from Example 2.1.

Each logical rule (1 , $\perp$, $\otimes$, $\wp$) has a corresponding transition axiom. These follow the prefix-continuation pattern $\pi.P \xrightarrow{\pi} P$, where performing the action π leads to the remainder of the process. Rule SYN allows for the pairing of concurrent actions via $l \mid l'$, which is used to facilitate communication by letting two endpoints connected by a restriction perform compatible actions. When two compatible actions are performed over endpoints connected by a restriction, rules $1\perp$ and $\otimes\wp$ simplify the term into an internal τ -transition, modeling the synchronisation of the session.

2.6 Process semantics and erasure

While the typing derivations of π LL dictate the valid interactions within a session, the underlying process terms possess a standard, untyped operational behavior that closely mirrors the classic π -calculus. To formally relate the typed derivations to their untyped execution, we follow [Montesi and Peressotti \[2021\]](#) and define an erasure function, $proc : Der \rightarrow Proc$, which strips away the typing information from a derivation. Recursively, $proc$ maps the head of a typing derivation to the specific process constructor it introduces, continuing structurally onto the process's continuation. For any well-typed π LL term, $proc(\mathcal{D})$ effectively extracts the pure process term from the derivation's conclusion.

$$\begin{array}{c}
 \frac{\frac{\frac{}{\vdash 0 :: \emptyset} \text{MIX}_0}{\vdash v[] . 0 :: v : 1} 1}{\vdash w().v[] . 0 :: v : 1, w : \perp} \perp \quad \frac{\frac{\frac{}{\vdash 0 :: \emptyset} \text{MIX}_0}{\vdash x[] . 0 :: x : 1} 1 \quad \frac{\frac{\frac{}{\vdash 0 :: \emptyset} \text{MIX}_0}{\vdash y[] . 0 :: y : 1} 1}{\vdash z().y[] . 0 :: y : 1, z : \perp} \perp}{\vdash x[] . 0 \mid z().y[] . 0 :: x : 1 \mid y : 1, z : \perp} \text{MIX} \\
 \frac{\vdash (w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0) :: v : 1, w : \perp \mid x : 1 \mid y : 1, z : \perp}{\vdash vxz(w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0) :: v : 1, w : \perp \mid y : 1} \text{MIX} \\
 \vdash vxz(w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0) :: v : 1, w : \perp \mid y : 1 \quad \text{CUT}
 \end{array}$$

Fig. 4. A derivation for the process $vxz(w().y[] . 0 \mid x[] . 0 \mid z().y[] . 0)$

Example 2.4. Consider the well-typed process P formed by adding a third parallel component to the derivation of [Example 2.2](#) before the cut, A derivation of which can be seen on [Figure 4](#):

$$P = vxz(w().v[] . 0 \mid x[] . 0 \mid z().y[] . 0)$$

The component containing w and v is independent from the component containing x , z , and y . This allows their respective actions to interleave freely, resulting in the following execution traces, illustrating the concurrency of the calculus:

$$\begin{array}{lll}
 P \xrightarrow{\tau} w() \xrightarrow{v[]} y[] \rightarrow 0 & P \xrightarrow{\tau} w() \xrightarrow{y[]} v[] \rightarrow 0 & P \xrightarrow{\tau} y[] \xrightarrow{w()} v[] \rightarrow 0 \\
 P \xrightarrow{w()} v[] \xrightarrow{\tau} y[] \rightarrow 0 & P \xrightarrow{w()} \tau \xrightarrow{v[]} y[] \rightarrow 0 & P \xrightarrow{w()} \tau \xrightarrow{y[]} v[] \rightarrow 0
 \end{array}$$

As formulated by [Montesi and Peressotti \[2021\]](#), the relationship between derivation transitions and process transitions is expressed through erasure. Given the projection $proc : Der \rightarrow Proc$, the process LTS enjoys erasure with respect to the derivation LTS when $proc$ is a homomorphism between the two transition systems. By [Montesi and Peressotti \[2021, Fact 4.6\]](#), this is equivalent to the relation $\{(proc(\mathcal{D}), \mathcal{D}) \mid \mathcal{D} \in Der\}$ being a strong bisimulation.

THEOREM 2.5 (ERASURE [MONTESI AND PERESSOTTI 2021, THEOREM 4.7]). *The process LTS, lts_p , enjoys erasure with respect to the derivation LTS, lts_d .*

Concretely, this correspondence means that, for any typing derivation $\mathcal{D}$ and label $l \in Lbl$:

- (1) if $\mathcal{D} \xrightarrow{l} \mathcal{D}'$, then $proc(\mathcal{D}) \xrightarrow{l} proc(\mathcal{D}')$; and
- (2) if $proc(\mathcal{D}) \xrightarrow{l} P'$, then there exists a derivation $\mathcal{D}'$ such that $\mathcal{D} \xrightarrow{l} \mathcal{D}'$ and $proc(\mathcal{D}') = P'$.

Because well-typed processes lie in the image of $proc$, erasure yields typability preservation:

COROLLARY 2.6 (TYPABILITY PRESERVATION [MONTESI AND PERESSOTTI 2021, COROLLARY 4.8]). *If P is well typed and $P \xrightarrow{l} P'$, then P' is well typed.*

$$\begin{array}{c}
\begin{array}{c}
x[x'].P \xrightarrow{x[x']} P \quad x(x').P \xrightarrow{x(x')} P \quad x[].P \xrightarrow{x[]} P \quad x().P \xrightarrow{x()} P \\
\frac{P \xrightarrow{l} P' \quad i(l) \cap f(Q) = \emptyset}{P \mid Q \xrightarrow{l} P' \mid Q} \text{PAR}_1 \quad \frac{Q \xrightarrow{l} Q' \quad i(l) \cap f(P) = \emptyset}{P \mid Q \xrightarrow{l} P \mid Q'} \text{PAR}_2 \\
\frac{P \xrightarrow{l} P' \quad Q \xrightarrow{l'} Q' \quad i(l \parallel l') \cap f(P \mid Q) = \emptyset}{P \mid Q \xrightarrow{l \parallel l'} P' \mid Q'} \text{SYN} \quad \frac{P \xrightarrow{x[x'] \parallel y(y')} P' \quad \text{v}xy P \xrightarrow{\tau} \text{v}xy \text{v}x'y' P'}{\text{v}xy P \xrightarrow{\tau} \text{v}xy P'} \otimes \otimes \\
\frac{P \xrightarrow{x[] \parallel y()} P' \quad \text{v}xy P \xrightarrow{\tau} P'}{\text{v}xy P \xrightarrow{\tau} P'} 1\bot \quad \frac{P \xrightarrow{l} P' \quad x, y \notin f(l) \cup i(l)}{\text{v}xy P \xrightarrow{l} \text{v}xy P'} \text{RES} \quad \frac{P =_{\alpha} Q \quad Q \xrightarrow{l} Q'}{P \xrightarrow{l} Q'} =_{\alpha}
\end{array}
\end{array}$$

$$\begin{array}{c}
i(l \parallel l') = i(l) \cup i(l') \\
f(l \parallel l') = f(l) \cup f(l') \\
i(\tau) = \emptyset \\
f(\tau) = \emptyset \\
i(x[]) = i(x()) = \emptyset \\
f(x[]) = f(x()) = \{x\} \\
i(x[x']) = i(x(x')) = \{x'\} \\
f(x[x']) = f(x(x')) = \{x\}
\end{array}$$

Fig. 5. π MLL transition rules for processes, as presented in Figure 4 of Montesi and Peressotti [2021].

Erasure therefore allows the process LTS to be extracted from the derivation-level semantics while preserving observable transition behaviour. The resulting SOS specification describes process evolution independently of typing derivations.

The process LTS states freshness and name-disjointness as explicit operational side conditions because it describes process evolution independently of typing derivations. For example, synchronising actions must not introduce the same name. This is expressed by the side condition $i(l) \cap i(l') = \emptyset$. Although corresponding side conditions in the derivation-level LTS are already enforced for well-typed derivations by the disjointness of hyperenvironment components, they are retained to support erasure to the process LTS [Montesi and Peressotti 2021, Remark 3.6]. The corresponding evolution of typing resources is described by the environment semantics introduced next.

2.7 Semantics of typing environments

Just as processes evolve through computation, the typing environments that govern them must also evolve to accurately track the state of a session. In π LL, types capture communication protocols. Consequently, the transition rules for typing environments specify exactly which actions are permitted and how the available resources are consumed by those actions.

Similar to how the erasure function was defined for processes, we again follow Montesi and Peressotti [2021] and define a function $env : Der \rightarrow Env$ that maps a valid typing derivation directly to the hyperenvironment in its conclusion (i.e., $env(\vdash P :: \mathcal{G}) = \mathcal{G}$). Using this extraction, Montesi and Peressotti [2021] derive a LTS specifically for typing environments (lts_e), shown in Figure 6.

In this LTS, observable actions actively consume resources from the environment. For example, if a hyperenvironment contains a waiting channel $x : \perp$, performing an input action $x()$ will consume this type, removing it from the resulting environment. Conversely, internal communications do not alter the available external resources. This is expressed by the τ -reflexivity axiom, $\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma$, which states that any valid hyperenvironment transitions to itself under a silent τ -action.

The rules in Figure 6 define which transitions are available at the environment level. For a well-typed process, both the process behaviour and resource evolution must step in unison such that whenever the process performs an action, its typing environment performs a corresponding transition. This correspondence is the property of **session fidelity**. Just as the *proc* projection extracts the process behaviour of a typed transition, the *env* projection extracts its corresponding environment evolution.

$$\begin{array}{c}
x:1 \xrightarrow{x[]} \emptyset \quad \Gamma, x:\perp \xrightarrow{x()} \Gamma \quad \Gamma, \Delta, x:A \otimes B \xrightarrow{x[x']} \Gamma, x':A \mid \Delta, x:B \quad \Gamma, x:A \wp B \xrightarrow{x(x')} \Gamma, x':A, x:B \\
\\
\frac{\mathcal{G} \xrightarrow{l} \mathcal{G}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G}' \mid \mathcal{H}} \text{PAR}_1 \quad \frac{\mathcal{H} \xrightarrow{l} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{l} \mathcal{G} \mid \mathcal{H}'} \text{PAR}_2 \quad \frac{\mathcal{G} \xrightarrow{l} \mathcal{G}' \quad \mathcal{H} \xrightarrow{l'} \mathcal{H}'}{\mathcal{G} \mid \mathcal{H} \xrightarrow{ll'} \mathcal{G}' \mid \mathcal{H}'} \text{SYN} \\
\\
\frac{\mathcal{G} \mid \Gamma, x:A^\perp \mid \Delta, y:A \xrightarrow{l} \mathcal{G}' \mid \Gamma', x:A^\perp \mid \Delta', y:A}{\mathcal{G} \mid \Gamma, \Delta \xrightarrow{l} \mathcal{G}' \mid \Gamma', \Delta'} \text{RES} \quad \frac{\mathcal{G} \mid x:1 \mid \Gamma, y:\perp \xrightarrow{x[]|y()} \mathcal{G} \mid \Gamma}{\mathcal{G} \mid \Gamma \xrightarrow{\tau} \mathcal{G} \mid \Gamma} 1\perp \\
\\
\frac{\mathcal{G} \mid \Gamma, \Delta, x:A \otimes B \mid \Xi, y:A^\perp \wp B^\perp \xrightarrow{x[x']|y(y')} \mathcal{G} \mid \Gamma, x:B \mid \Delta, x':A \mid \Xi, y:B^\perp, y':A^\perp}{\mathcal{G} \mid \Gamma, \Delta, \Xi \xrightarrow{\tau} \mathcal{G} \mid \Gamma, \Delta, \Xi} \otimes \wp
\end{array}$$

Fig. 6. π MLL, transition rules for typing environments, as presented in Figure 5 of [Montesi and Peressotti 2021].

The following is Corollary 4.10 of Montesi and Peressotti [2021], stated concretely for a well-typed process:

COROLLARY 2.7 (SESSION FIDELITY FOR PROCESSES). *Let $\mathcal{D}$ be a typing derivation such that $\text{proc}(\mathcal{D}) = P$ and $\text{env}(\mathcal{D}) = \mathcal{G}$. If $P \xrightarrow{l} P'$, then there exist a hyperenvironment $\mathcal{G}'$ and a typing derivation $\mathcal{D}'$ such that $\mathcal{G} \xrightarrow{l} \mathcal{G}'$, $\text{proc}(\mathcal{D}') = P'$ and $\text{env}(\mathcal{D}') = \mathcal{G}'$.*

To illustrate session fidelity, consider a particular execution of a well-typed process and the corresponding evolution of its typing hyperenvironment.

Example 2.8 (Session fidelity along an execution). Recall the process derivation from Example 2.4, initially typed by the hyperenvironment $\mathcal{G} = v:1, w:\perp \mid y:1$. Consider the process execution in which the observable input on w is followed by the internal synchronisation on the restricted session, and then by the observable outputs on y and v . Session fidelity requires each of these process transitions to be matched by a transition of the typing hyperenvironment:

$$v:1, w:\perp \mid y:1 \xrightarrow{w()} v:1 \mid y:1 \xrightarrow{\tau} v:1 \mid y:1 \xrightarrow{y[]} v:1 \xrightarrow{v[]} \emptyset.$$

The input action on w consumes the resource $w:\perp$. After this transition, the environment LTS allows the output action $y[]$, since $y:1$ is already present, although the process cannot yet perform that action. In the process, the corresponding output remains guarded until the internal synchronisation on the restricted session has occurred. This τ -transition does not affect the externally visible resources, but exposes the output on y , whose transition consumes $y:1$. The final output on v then consumes the remaining external resource.

2.8 Managing bound variables

Having introduced the operational semantics of π LL and the session-fidelity property it supports, we now address the treatment of explicitly representing bound names, scopes, and freshness conditions.

Several constructs in π LL introduce scoped identifiers. Restriction binds pairs of session endpoints, input binds received channel names, polymorphic communication binds type variables, and duplication introduces a fresh channel endpoint in its continuation. These binding constructs occur throughout the process syntax and the labelled transition system.

For bound channel names, the concrete representative chosen for a binder is semantically irrelevant provided that the binding structure is preserved. For example, renaming the two restricted

endpoints of a session does not alter the behaviour of a process. However, a formal account of substitution, transitions, and typing derivations must track names, scopes, and freshness conditions explicitly.

This principle is expressed by α -equivalence, which identifies process terms that differ only in their choice of bound channel names. For instance, the following two processes are α -equivalent:

$$\mathbf{v}ab(a[\cdot].0 \mid b().x[\cdot].0) =_{\alpha} \mathbf{v}zw(z[\cdot].0 \mid w().x[\cdot].0)$$

The operational semantics of π LL uses α -equivalence through $\text{rule } =_{\alpha}$, allowing bound names to be changed before a transition when necessary to avoid capture.

Substitution exposes the same issue. Consider replacing the free name x by z in the right-hand process from the preceding example:

$$\mathbf{v}zw(z[\cdot].0 \mid w().x[\cdot].0)$$

Here, z is already bound by the restriction, so naively replacing x with z would cause the restriction to capture the newly inserted z .

Since restriction binds session endpoints as pairs, both the bound name z and w must be α -renamed to fresh names, such as to a and b , before the substitution proceeds. Generally, any binder capturing a substituted free name must be renamed before substitution occurs.

In paper-based presentations, this issue is commonly handled using *Barendregt's variable convention*, under which bound names may be assumed fresh for the free names and other objects relevant to the current argument. This convention keeps paper proofs readable, but it remains an informal meta-level assumption.

A proof assistant requires each relevant freshness condition and renaming step to be represented and verified explicitly [Aydemir et al. 2005]. A direct mechanisation using named binders would therefore introduce substantial proof overhead for capture avoidance and reasoning up to α -equivalence. The formalisation instead adopts a locally nameless binding architecture, retaining explicit free identifiers while representing bound occurrences by indices. This avoids explicit reasoning up to α -equivalence for bound occurrences, while relocating the remaining binder-related obligations to opening, local closure, freshness, and substitution. The following chapter develops this architecture in detail.

3 Binding architecture

This chapter develops the locally nameless binding architecture used throughout the formalisation. We begin by stating the design goals that motivate the approach (Section 3.1), then introduce the locally nameless representation and its core operations (Sections 3.2 and 3.3), and conclude with co-finite quantification (Section 3.4), which governs how inductive definitions interact with binders.

3.1 Design goals and the approach

This chapter introduces the binding representation used throughout the formalisation. The representation is motivated by three primary design goals:

- (1) **Eliminate α -equivalence overhead.** Binder structures that differ only in the names chosen for bound occurrences should be represented identically, avoiding explicit reasoning about α -equivalence within the proof assistant.
- (2) **Preserve human readability.** Free identifiers should remain explicit in process terms, typing environments, and transition labels, so that the mechanised definitions and proof states remain recognisable relative to the paper-based presentation.
- (3) **Avoid informal meta-level conventions.** The representation should not rely on Barendregt's convention or unstated freshness assumptions. Capture avoidance should follow

from the representation of bound occurrences, while any remaining freshness requirements should appear explicitly in definitions or proof obligations.

Representation alternatives. A direct representation using named bound variables would remain closest to the paper syntax. However, it would require the mechanisation to define and reason explicitly about capture-avoiding renaming and α -equivalence whenever a transition or substitution passes through a binder. At the opposite extreme, a fully nameless representation could encode channel references solely through indices, making equality of binder structures direct but obscuring the free channel names that occur explicitly in typing hyperenvironments and transition labels. Following [Charguéraud \[2012\]](#), the formalisation instead adopts a locally nameless architecture: free identifiers remain explicit, while bound occurrences are represented by indices. Consequently, binder structures that differ only by the choice of bound names coincide structurally, while the observable names used by the operational semantics remain explicit.

Although both channel names and type variables involve binding, they have different roles in the calculus. Channel names participate in dynamic communication and occur in transition labels and typing environments and their binders are therefore opened with explicit fresh names. Type variables remain statically scoped within session types and typing derivations, and their binders are handled through a depth-indexed treatment based on shifting, local closure, and substitution. The formalisation consequently applies the same free/bound separation to both categories, while adapting the associated proof infrastructure to their respective semantic roles.

The remainder of this chapter develops the operations required by this representation. In place of explicit α -equivalence reasoning, the formalisation must establish properties of opening, closing, local closure, and freshness. These obligations are subsequently reflected in the typing relation and in the metatheoretic proofs.

3.2 The locally nameless representation

The locally nameless representation, as presented by [Charguéraud \[2012\]](#), separates variables into two disjoint syntactic categories: free names and bound indices. This combines the readability of named syntax for free variables with a structural representation of binding.

- (1) **Bound variables** are represented using *de Bruijn indices* [[de Bruijn 1972](#)], where each index denotes the distance to its corresponding binder. Since the representation is purely positional, α -equivalent terms become structurally identical.
- (2) **Free variables** retain explicit names, preserving the readability of open terms and allowing standard name-based reasoning about environments and transition labels.

By structurally separating these two concepts, the locally nameless approach simplifies standard operations such as capture-avoiding substitution. Since free variables and bound variables inhabit distinct syntactic categories, substitution on free variables is defined to not interact with bound indices. We apply this theory to both the structural and operational layers of π LL.

LN syntax for types. In the original presentation of π LL, polymorphic types rely on standard named variables. Under the locally nameless architecture, we expand the syntax of type variables to explicitly distinguish between free and bound occurrences. The grammar for types is updated as follows:

$$\begin{aligned} T &::= \text{free}(\alpha) \mid \text{bound}(k) && (\text{where } \alpha, k \in \mathbb{N}) \\ A, B &::= \cdots \mid \exists.A \mid \forall.A \mid X \mid X^\perp && (\text{where } X \in T) \end{aligned}$$

Here, a free type variable carries a unique identifier α , while a bound type variable carries a de Bruijn index k . For example, the polymorphic type $\forall X. \exists Y. (X \wp Y)$ is structurally represented

without names using the new syntax as $\forall.\exists.(\text{bound}(1) \wp \text{bound}(0))$, where 0 points to the inner existential binder and 1 points to the outer universal binder.

LN syntax for processes. Similarly, the standard process syntax displayed in [Section 2.2](#) is updated to reflect the locally nameless separation of channel endpoints. A channel is split into its bound and free counterparts, where bound channel names are represented by indices, while free channels remain explicit identifiers. This split reflects a semantic distinction: free channel names are observable, appearing in transition labels and typing environments, while bound names are purely structural. Renaming a bound variable never changes the behaviour of a process, whereas renaming a free channel can alter which sessions it participates in. The locally nameless representation makes this distinction explicit by construction.

Naively removing the bound names from the process syntax would turn binding operations such as $x[y].P$ and $x(y).P$ into $x[].P$ and $x().P$ making them syntactically indistinguishable from their non-binding unit counterparts. To combat this we annotate anonymous binders using placeholder symbols. We use $\bullet$ for a bound process name, and $\diamond$ for a bound type variable.

$$\begin{aligned} \text{Channel} &::= \text{free}(x) \mid \text{bound}(k) & (\text{where } x, k \in \mathbb{N}) \\ P, Q &::= \dots \mid x[\bullet].P \mid x(\bullet).P \mid \nu(\bullet, \bullet)P \mid x(\diamond).P & (\text{where } x \in \text{Channel}) \end{aligned}$$

While the visual anonymity of these constructors is evident from the grammar, their mechanical implementation relies on precise index management. For a multi-name binder like restriction, $\nu(\bullet, \bullet)P$, the two dual endpoints are simultaneously mapped to distinct indices within the continuation of P , namely 0 and 1. For communication prefixes like $x(\bullet).P$, the transmitted name is implicitly bound as index 0 within P , while the explicit free channel x is deliberately preserved as the subject. By maintaining explicit free names strictly for these outward-facing communications, the transition labels and operational semantics of the calculus remain completely unaffected by the architectural shift.

While the locally nameless architecture avoids explicit reasoning up to α -equivalence for bound occurrences, it requires specialised operations for moving between abstracted and instantiated forms. In particular, substitution, typing, and transitions under binders depend on formally defining opening and closing operations.

3.3 Opening and closing binders

Because the locally nameless representation separates free and bound variables, standard substitution cannot be applied uniformly under binders. Instead, moving between bound and free representations is handled by two dual operations, namely *opening* and *closing* [[Charguéraud 2012](#)].

Opening (Instantiation). Opening is the operation of traversing into a binding construct and replacing the exposed bound index with a concrete free name. Formally, opening a process P at depth k with a free name x is denoted $\{k \mapsto x\}P$.

The operation proceeds via structural recursion. A depth counter, initially set to 0, tracks the number of binders traversed. As the recursion descends through the syntax tree, the counter increments when passing a new binder. Consider being at depth k , then traversing a single-name binder like $x(\bullet).P$ increments the depth to $k+1$, while traversing a double-name binder like $\nu(\bullet, \bullet)P$ would increment it to $k+2$.

When the recursion reaches a bound variable $\text{bound}(n)$, the index n is compared against the current depth k . If $n = k$, the variable refers to the binder currently being opened and is replaced by the free name $\text{free}(x)$. Otherwise, the index refers to a different binder and is left unchanged.

When the opening operation encounters an existing free name $\text{free}(y)$, it leaves that occurrence unchanged, and only targets the bound occurrence at the relevant index, replacing them with the supplied free name. Since binders are represented by indices rather than names, this replacement does not require α -renaming to prevent a newly inserted free name from being captured by a named binder. Opening alone does not, however, ensure that the chosen name is fresh for the free names already occurring in the continuation or its typing environment. These freshness and distinctness requirements are instead imposed by side conditions of the typing and transition relations.

In the operational semantics, a continuation under a single channel binder, such as the input prefix $z(\bullet).P$, is instantiated with a concrete channel name x by opening P as $\{0 \mapsto x\}P$. For a restriction $\nu(\bullet, \bullet)P$, the two bound endpoint occurrences are instantiated with channel names x and y using the corresponding two-binder opening operation, written here as $\{0 \mapsto x\}(\{1 \mapsto y\}P)$.

Closing (abstraction). Closing is the companion abstraction operation to opening. It is the operation of taking an open term and abstracting a specific free name x into a bound index at depth k , denoted $\{k \leftarrow x\}P$. While recursing, bound indices are ignored as they cannot contain free names. When the recursion encounters a free term like $\text{free}(y)$, it is checked if $y = x$. If they match, the free name is replaced by $\text{bound}(k)$.

This operation is primarily used when constructing new binders, such as moving a free channel into a restricted scope. To close multiple free names simultaneously, such as abstracting x and y into a restriction, the operations are applied sequentially, by first closing y at depth $k + 1$, followed by closing x at depth k .

Locally closed terms. The separation of names and indices introduces a new syntactic anomaly, namely that a term can contain a *dangling* index that does not point to any enclosing binder. For example, the process term $\text{bound}(0)[].\mathbf{0}$ is syntactically valid, matching the syntax for an empty send, but mathematically it is meaningless because the index 0 has no corresponding binder.

To exclude such terms, the locally nameless representation introduces the predicate of being *locally closed*, denoted $lc(P)$. A term is locally closed if and only if all of its de Bruijn indices are bound by enclosing constructors. Local closure therefore characterises the well-scoped fragment of the process syntax.

This predicate forms a key invariant throughout the mechanisation. Operations that manipulate syntax under binders, such as substitution and transitions, must either be shown to preserve local closure directly or be used in a context where local closure can be recovered. In the typing development, this connection is established through regularity: every valid typing derivation produces a locally closed process. Consequently, later arguments about typed transitions may obtain well-scopedness from typing rather than reproving it independently for each intermediate process.

3.4 Co-finite quantification

With the syntax partitioned into names and indices, and the mechanics of opening and closing established, the remaining design question is how binder-sensitive inductive relations, in particular typing judgements, should expose fresh names.

In paper-based proofs, when a rule requires opening a binder, Barendregt's Variable Convention allows the author to state: *pick a fresh name x* . In a proof assistant, this informal choice must instead be represented explicitly in the constructors of the relevant inductive relations. Two standard formulations introduce difficulties for mechanised proofs [Aydemir et al. 2008; Charguéraud 2012]:

- (1) **Opening with one fresh name.** A rule requires that its premise is valid only for one particular fresh name when the derivation is constructed. This is convenient when applying

the rule, but yields an induction hypothesis tied to that particular name, even when a later proof requires a different fresh name.

- (2) **Opening with every name.** Alternatively, a rule may require that its premise is valid for every name. This yields a strong general induction hypothesis, but makes the rule hard to apply as the premise cannot generally be established for names that already occur free in the process or its surrounding context.

Following Aydemir et al. [2008]; Charguéraud [2012], the formalisation uses co-finite quantification for typing rules that open process binders with fresh channel names. Rather than fixing one particular fresh representative, a constructor records a finite exclusion set L and requires its premise for every name outside that set. Schematically, a rule introducing one fresh channel uses a premise of the form $\forall x \notin L$, while a rule introducing two fresh channels additionally requires the selected names to be distinct.

Choice of freshness discipline. Co-finite quantification occupies the middle ground between the two formulations above. It introduces explicit finite exclusion sets into binder-sensitive constructors, but provides an induction principle in which a proof may choose a name outside both the exclusion set recorded by the rule and any additional finite set of names relevant to the current argument. In the present formalisation, this is used for process binders whose continuations are instantiated with concrete free channel names. For example, a proof concerning an opened continuation may select a channel name that is simultaneously fresh for the process, its typing environment, and any labels involved in the argument.

Application to π LL types. Although type variables are represented using the same separation between free and bound occurrences, their metatheory is handled using depth-indexed reasoning rather than co-finite quantification. Type variables remain statically scoped within types and typing derivations rather than being communicated as observable channel names, so introducing finite freshness sets for them would add proof infrastructure not otherwise required by their role in the calculus.

Instead, polymorphic typing rules are formulated using this depth-indexed approach based on a de Bruijn shifting operator. We define the shifting of a type A at cutoff depth d by distance c , denoted mathematically as $\uparrow_d^c A$. Using this infrastructure, consider the typing rule for universal type input:

$$\frac{n+1 \vdash P :: \uparrow_0^1 \Gamma, x : B}{n \vdash x(\diamond).P :: \Gamma, x : \forall B} \mathcal{V}_{\text{DB}}$$

Rather than opening the binder with a fresh explicit type variable, the typing judgement increments the depth counter to $n+1$, bringing index 0 into scope. The surrounding typing environment is shifted by distance 1 at cutoff 0, written $\uparrow_0^1 \Gamma$, so that type-variable occurrences already present in the environment retain their intended binding structure under the newly introduced binder.

Application to π LL channels. Channel binders are treated differently because they occur in the dynamic process syntax and are opened with concrete free channel names. For example, consider the typing rule introducing the input prefix $x(\bullet).P$. In the paper-based presentation, the premise implicitly relies on a fresh session name y . In the mechanised presentation, this freshness requirement is expressed by co-finite quantification together with the opening operation:

$$\frac{\forall y \notin L, \quad n \vdash \{0 \mapsto y\}P :: \Gamma, x : B, y : A}{n \vdash x(\bullet).P :: \Gamma, x : A \wp B} \mathcal{W}_{\text{LN}}$$

Here, the rule states that the process is well typed when, for every fresh channel name y outside a finite exclusion set L , the continuation opened with y is well typed under the typing environment extended with that name. The formalisation therefore uses a mixed binding strategy. Namely, co-finite opening for dynamic channel binders and depth-indexed reasoning for static type-variable binders.

This mixed strategy determines where the remaining proof overhead appears. Depth-indexed treatment of polymorphic types avoids managing fresh type-variable sets, but requires explicit shifting and weakening lemmas when typing derivations cross polymorphic binders. Co-finite quantification for process channels avoids explicit α -equivalence reasoning and provides strong induction hypotheses, but later requires bridge lemmas when an operational transition chooses concrete fresh names. These costs are reflected in the mechanised definitions and proofs developed in the following chapters.

4 Formalisation

This section describes the mechanisation of π LL in Lean 4, bridging the theoretical foundations developed in [Sections 2 and 3](#) and their concrete implementation. The formalisation is organised into three top-level namespaces: `PiLL.Model`, which defines the static structure of the calculus, `PiLL.Semantics`, which defines its operational behaviour, and `PiLL.SessionFidelity`, which establishes the metatheoretic results connecting the two. Each component is presented in dependency order, highlighting design decisions and divergences from the presentation of [Montesi and Peressotti \[2021\]](#).

4.1 Model

The `Model` namespace formalises the static structure of the full π LL calculus and is organised around the layered architecture introduced in [Section 2](#). The mechanisation proceeds bottom-up: session types, defined in `STypes/`, provide the binding structure used by processes in `Processes/`, processes are then typed relative to environments and hyperenvironments from `Environments/` and `HyperEnvironment/`, finally, typing derivations and their metatheoretic infrastructure are collected in `Judgement/`. This includes inversion lemmas, linearity properties, substitution principles, and the co-finite freshness machinery described in [Section 3.4](#). Shared syntactic infrastructure used across these layers is provided by `Base.lean`. An overview of the file structure can be found in [Listing 2](#).

4.1.1 Session types. Session types are defined in `STypes/Basic.lean` as the inductive datatype `Types`, representing raw session-type expressions. Well-scoped use of bound type variables is characterised separately through local closure; this condition is incorporated into the mechanised typing judgement introduced in [Section 4.1.5](#) and later shown to be preserved by typing derivations. Following the representation strategy established in [Section 3](#), type variables are isolated into a separate inductive type, `TVar`, which explicitly distinguishes between free and bound occurrences:

```
inductive TVar : Type
| free (x : FTVar)
| bound (x : BTVar)
```

Here, `FTVar` represents a free type variable identified by a natural number, while `BTVar` represents a bound type variable encoded as a de Bruijn index.

The remaining constructors directly mirror the theoretical grammar of the calculus. In addition, the formalisation introduces uninterpreted atomic types (`atom` and `atomDual`), representing protocol leaves not further decomposed by the session type grammar and providing the base cases for the

inductive structure of the syntax. Table 3 illustrates a representative correspondence between the paper syntax, the locally nameless representation, and the Lean implementation.

Table 3. Comparison of standard, locally nameless, and Lean representations for π LL types.

Standard	Locally Nameless	Lean Constructor	Binds
$A \otimes B$	$A \otimes B$	tensor (A B : Types)	None
1	1	one	None
$\forall X.A$	$\forall.A$	forall_ (A : Types)	1 type
$\exists X.A$	$\exists.A$	exists_ (A : Types)	1 type
X	X	var (v : TVar)	None

Notably, the polymorphic binders $\forall.A$ and $\exists.A$ reflect the depth-indexed approach detailed in Section 3.4. As shown in Section D.1, the corresponding Lean constructors keep the bound de Bruijn index implicit, taking only the continuation body A as an argument. An underscore “_” is appended to the constructor names to prevent namespace collisions with Lean’s built-in `forall` and `exists` keywords.

Duality. `Types.dual` implements duality as a structurally recursive function, mapping each constructor to its dual. For example:

```
def Types.dual : Types → Types
| .atom a      => .atomDual a
| .atomDual a  => .atom a
| .tensor A B  => .parr   (dual A) (dual B)
| .forall_ A   => .exists_ (dual A)
| .one         => .bot
| ...
```

The fundamental property that duality is an involution, $(A^\perp)^\perp = A$, is proved in `STypes/Lemmas.lean` by structural induction.

The formalisation additionally establishes that duality preserves local closure and commutes with both shifting and substitution:

$$A.\text{lc} \iff A^\perp.\text{lc} \quad \uparrow_d^c (A^\perp) = (\uparrow_d^c A)^\perp \quad B^\perp\{A/k\} = (B\{A/k\})^\perp$$

Local closure. `STypes/Properties.lean` defines local closure for types as a depth-indexed predicate `Types.lc (k : Nat) : Types → Prop`, where the depth parameter k tracks how many binders the recursion has traversed.

At the leaves of the syntax tree, local closure is evaluated by the helper predicate `TVar.lc`. Because free variables are explicit they do not contain any indices, as such they are inherently locally closed at any depth. By contrast, a bound variable is only locally closed if its de Bruijn index i is strictly less than the current depth k , ensuring it points to an enclosing scope:

```
def TVar.lc : Nat → TVar → Prop
| _, .free _   => True
| k, .bound i  => i < k
```

The primary `Types.lc` predicate is then defined by structural recursion. For non-binding constructors, the depth parameter distributes across the sub-terms. When a variable is encountered, the predicate delegates the check to `TVar.lc`. When descending into the body of a polymorphic binder, the depth counter increments by one, bringing the newly introduced index safely into scope:

```
def Types.lc : Nat → Types → Prop
| _, .one          => True
| k, .var v        => TVar.lc k v
| k, .tensor A B   => A.lc k ∧ B.lc k
| k, .forall_ A    => A.lc (k + 1)
| k, .exists_ A    => A.lc (k + 1)
| ...
```

Finally, a type is defined as *closed* if it contains no dangling indices at the top level, meaning it is locally closed at depth 0.

Shifting. `STypes/Shift.lean` defines shifting following the standard de Bruijn shifting operation introduced in [Section 3.4](#). The operation traverses the syntax tree structurally using a cutoff depth d and a shift amount c .

For standard connectives, shifting distributes recursively across subterms, while base constructors such as `one` and `bot` remain unchanged. When traversing a polymorphic binder like `forall_`, the cutoff depth increments by one to account for the newly entered scope:

```
def Types.shift (d c : Nat) : Types → Types
| .var v          => .var (v.shift d c)
| .tensor A B     => .tensor (A.shift d c) (B.shift d c)
| .forall_ A      => .forall_ (A.shift (d + 1) c)
| t              => t
| ...
```

At the variable level, free variables remain unchanged. For bound indices, the function compares the de Bruijn index i against the cutoff depth d . Indices satisfying $i < d$ are locally bound within the traversed scope and therefore remain unchanged. Otherwise, the index is incremented by c to preserve its reference to the surrounding environment:

```
def TVar.shift (d c : Nat) : TVar → TVar
| .bound i => if i < d then .bound i else .bound (i + c)
| .free n  => .free n
```

The operation is exposed through the common `HasShift` interface, allowing the notation $A \uparrow d, c$, $A \uparrow c$ and A^+ to be used uniformly for types and, later, for processes, environments, and hyperenvironments. The shared notation declarations are given in [Listing 3](#).

Substitution. `STypes/Subst.lean` defines substitution by replacing a target de Bruijn index k in a host type B with a type A , denoted $B\{A/k\}$. At the variable level, the function compares a bound de Bruijn index i with the target depth k . If $i = k$, the target index has been reached and the variable is replaced by A . If $i < k$, the variable is bound by an inner scope and is left unchanged. Finally, if $i > k$, the index refers to an outer binder beyond the substituted variable. Since substitution removes the binder corresponding to k , such indices are decremented by one to preserve their relative distance to the surrounding environment.

```
def Types.subst (B A : Types) (k : Nat) : Types :=
  match B with
  | .var (.free n) => .var (.free n)
```

```

| .var (.bound i) =>
  if i == k then A
  else if i > k then .var (.bound (i - 1))
  else .var (.bound i)
| .tensor A B => .tensor (A.subst A k) (B.subst A k)

```

The operation is lifted to the full Types syntax by structural recursion. For ordinary connectives, substitution distributes across subterms. When descending into a polymorphic binder such as `forall_`, the target depth is incremented to $k + 1$, and the substituting type A is shifted by one. This prevents the free indices in A from being captured by the newly entered binder:

```

| .forall_ B => .forall_ (B.subst (shift 0 1 A) (k + 1))

```

The interaction between substitution and local closure is captured by `Types.lc_subst_inv`. Given a type A that is locally closed at depth $n + k$, the lemma relates local closure before and after substituting A for index k in B : $(B\{A/k\}).lc(n+k) \iff B.lc(n+k+1)$. Intuitively, substituting for index k essentially peels off a type binder, as such local closure at depth $n + k$ after substitution corresponds to local closure at depth $n + k + 1$ before substitution. A specialisation of this result is used later to show that typing judgements contain locally closed syntax.

As with shifting, substitution is exposed through the common `HasSubst` interface, allowing the notation $B\{A/k\}$ to be reused when substitution is lifted to processes, environments, and hyperenvironments (see [Listing 3](#)).

Notation. `STypes/Notation.lean` introduces notation mirroring the mathematical presentation from [Section 2.3](#). The only divergence is that exponentials are written $!!A$ and $??A$ to avoid clashes with Lean syntax. The corresponding Lean notation declarations are given in [Listing 4](#).

4.1.2 Processes. Processes are defined in `Processes/Basic.lean` as the inductive datatype `Proc`, representing raw process terms independently of whether they are well typed. Following the established locally nameless architecture, channel references are partitioned into a dedicated `Channel` inductive. Explicit free channels are instantiated as concrete natural numbers (`FPName`). While the raw process syntax permits arbitrary reuse of these numbers, using `Nat` allows the formalisation to systematically compute strictly fresh names by incrementing the supremum of any finite set of existing names, as mechanised in `Processes/Fresh.lean`. Conversely, bound channels are represented using de Bruijn indices, encoding their relative distance to the corresponding binder.

```

inductive Channel : Type where
| free  (x : FPName)
| bound (x : BPName)

```

As discussed in [Section 3.2](#), binding constructs in the process syntax no longer carry explicit bound channel names. Instead, bound occurrences are captured purely by their relative position. Comparing the standard paper grammar with the intermediate locally nameless notation and the final constructors of the inductive `Proc` type illustrates this abstraction:

Standard	Locally Nameless	Lean Constructor	Binds
$x[y].P$	$x[\bullet].P$	tensor (x : Channel) (P : Proc)	1 channel
$x(y).P$	$x(\bullet).P$	parr (x : Channel) (P : Proc)	1 channel
$\nu xy.P$	$\nu(\bullet, \bullet).P$	cut (P : Proc)	2 channels
$x(X).P$	$x(\diamond).P$	input (x : Channel) (P : Proc)	1 type
$x[\text{DUP}](x').P$	$x[\text{DUP}](\bullet).P$	duplicate (x : Channel) (P : Proc)	1 channel

Notice that cut takes only a continuation P with no explicit channel arguments, as both dual endpoints are simultaneously bound as indices 0 and 1 within the scope of P . Similarly, for constructs like tensor, parr, and input, the subject channel x over which the interaction occurs is kept explicit, while the name or type variable bound in the continuation is anonymised and represented by an index. The full inductive definition is listed in [Section D.2](#).

Opening and closing. Processes/OpenClose.lean defines opening and closing operations following the locally nameless principles established in [Section 3.3](#). In the theoretical presentation, opening a process at depth k with x is denoted as $\{k \mapsto x\}P$, and conversely, abstracting a specific name back into a bound index is denoted as $\{k \leftarrow x\}P$.

To mechanise this in Lean, we denote explicit free channels as $\#x$ (mapping to `Channel.free x`) and introduce custom notation for these substitution operations. Expressions such as $P(\langle k | \#x \rangle)$ and $P(\langle k | \#x, \#y \rangle)$ denote the opening of a process by instantiating the k -th (and $k + 1$ -th) bound indices with free channels. Conversely, $P\langle\langle k | \#x \rangle\rangle$ and $P\langle\langle k | \#x, \#y \rangle\rangle$ denote the closing of a process by abstracting x and y back into de Bruijn indices. For convenience, the depth parameter can be omitted when operating at the innermost binder ($k = 0$), simplifying the notation to $P(\langle \#x, \#y \rangle)$ and $P\langle\langle \#x, \#y \rangle\rangle$. The typeclasses and notation declarations supporting these overloaded operations are collected in [Section C](#).

Because binders are represented positionally, the depth parameter k increments according to the number of channels bound by each constructor. For standard communication prefixes like tensor, parr, and the server duplicate operation, exactly one channel is bound, incrementing k by one. The restriction constructor (cut) simultaneously binds two dual endpoints, requiring k to increment by two. The input constructor binds a type variable rather than a channel and therefore leaves k unchanged. The server constructor additionally carries an explicit set of dependency channels $zs : \text{Finset Channel}$, representing the replicable processes the server depends on, which must be acted on simultaneously with x and P .

```

def Proc.open (P : Proc) (u : Channel) (k : Nat) : Proc :=
  match P with
  | .tensor x P   => .tensor (x.open u k) (P.open u (k + 1))
  | .parr x P     => .parr (x.open u k) (P.open u (k + 1))
  | .cut P        => .cut (P.open u (k + 2))
  | .duplicate x P => .duplicate (x.open u k) (P.open u (k + 1))
  | .server x zs P => .server (x.open u k) (zs.open u k) (P.open u k)
  | .input x P    => .input (x.open u k) (P.open u k)
  | ...

```

For channel occurrences, the recursive traversal delegates to `Channel.open`, for the dependency set carried by a server, it uses the pointwise lifting `Finset.open`. For bound channels, the de Bruijn index i is compared against the target depth k and replaced with v upon an exact match. For

dependency sets, `Channel.open` is mapped pointwise. In both cases free channels remain entirely unaffected:

```
def Channel.open (u v : Channel) (k : Nat) : Channel :=
  match u with
  | bound i => if i == k then v else bound i
  | c => c

def Finset.open (zs : Finset Channel) (v : Channel) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.open v k)
```

For restriction, the two-name opening interface is instantiated by sequential opening, replacing index $k + 1$ with the second endpoint v , then replacing index k with the first endpoint u :

```
instance : HasOpenTwo Proc Channel Channel Nat where
  open_ P u v k := (Proc.open (Proc.open P v (k + 1)) u k)
```

To complete the bidirectional infrastructure, `Proc.close`, `Channel.close`, and `Finset.close` define the corresponding closing operation. `Proc.close` traverses the process tree using a structural recursion identical to `Proc.open`, applying the same depth-incrementing rules. At the leaves, `Channel.close` compares each free channel against the target name x and abstracts it into a bound de Bruijn index at depth k upon a match, while bound indices are left unchanged. Dependency sets are handled pointwise by `Finset.close`:

```
def Channel.close (u : Channel) (x : FPNName) (k : Nat) : Channel :=
  match u with
  | .free name => if x == name then .bound k else .free name
  | .bound i   => .bound i

def Finset.close (zs : Finset Channel) (x : FPNName) (k : Nat) : Finset Channel :=
  zs.image (fun u => u.close x k)
```

Local closure. `Processes/LocalClosure.lean` defines local closure for processes as a two-parameter predicate `Proc.lc (k n : Nat) : Proc → Prop`. Because the calculus features a two-sorted syntax, the predicate must maintain independent depth counters: k tracks the depth of channel binders, while n tracks the depth of type binders. The latter is required because processes may contain embedded types whose free indices must be interpreted relative to the surrounding type-binder depth.

The structural recursion evaluates each constructor by incrementing the appropriate depth counter according to the number and sort of binders introduced. Channel binders increase the channel depth k , while type binders increase the type depth n . For example, tensor binds a single continuation channel, incrementing the depth to $k + 1$, whereas cut binds two dual endpoints simultaneously, incrementing the depth to $k + 2$. Conversely, input binds a type variable, leaving k unchanged while incrementing n to $n + 1$. The server constructor therefore requires local closure of x , zs , and P . Constructors introducing no binders leave both counters unchanged.

```
def Proc.lc : Nat → Nat → Proc → Prop
| _, _, .nil           => True
| k, n, .one x P       => x.lc k ∧ P.lc k n
| k, n, .tensor x P    => x.lc k ∧ P.lc (k + 1) n
| k, n, .cut P         => P.lc (k + 2) n
| k, n, .input x P     => x.lc k ∧ P.lc k (n + 1)
```

```
| k, n, .server x zs P => x.lc k ∧ zs.lc k ∧ P.lc k n
| ...
```

At the leaves, `Channel.lc` checks that any bound index is strictly within the current depth, while free channels are trivially locally closed. For dependency sets, `Finset.lc` requires every channel in the set to satisfy `Channel.lc k`:

```
def Channel.lc : Nat → Channel → Prop
| _, .free _ => True
| k, .bound i => i < k

def Finset.lc (zs : Finset Channel) (k : Nat) : Prop :=
  ∀ z ∈ zs, z.lc k
```

A process is considered globally *closed* when it contains no dangling indices of either sort, meaning `Proc.lc 0 0` holds.

Free names and freshness. `Processes/Names.lean` defines `Proc.f : Proc → Finset FPNName` to compute the free channel names of a process. The function traverses the syntax tree structurally, collecting all explicit free channel identifiers into a finite set. Since locally nameless binders are represented using anonymous indices, bound channels contribute nothing to this set.

For constructors carrying explicit subject channels, such as `tensor` and `link`, the free names of those channels are combined with the recursively computed free names of their continuations. Constructors without explicit channel identifiers of their own, such as `cut`, contribute only the free names recursively obtained from their continuations. The `server` constructor contributes the free names of its subject channel, its dependency set and continuation:

```
def Proc.f : Proc → Finset FPNName
| .cut P          => P.f
| .par P Q        => P.f ∪ Q.f
| .link x y       => x.f ∪ y.f
| .tensor x P     => x.f ∪ P.f
| .one x P        => x.f ∪ P.f
| .server x zs P  => x.f ∪ zs.f ∪ P.f
| .nil           => {}
| ...
```

At the leaves, `Channel.f` returns the singleton set for free channels and the empty set for bound indices. For dependency sets, `Finset.f` collects free names across all channels in the set:

```
def Channel.f : Channel → Finset FPNName
| .free x => {x}
| .bound _ => {}

def Finset.f (zs : Finset Channel) : Finset FPNName :=
  zs.biUnion Channel.f
```

The resulting finite set is used by `Processes/Fresh.lean` to generate names outside a forbidden context. Since free process names are natural numbers, a fresh name for a finite set L is computed as `freshName L = sup(L) + 1`, with `fresh_is_fresh` confirming that every element of L is strictly smaller than this value. The lemmas `exists_one_fresh` and `exists_two_fresh` then guarantee one or two distinct names outside L , which becomes relevant later when having to instantiate

co-finite quantification premises, in particular for terms like restriction, where two fresh and distinct endpoints must be introduced simultaneously.

Substitution. The process syntax supports two substitution operations, corresponding to the occurrence of both channel names and session types in process terms. Name substitution is defined in `Processes/SubstNames.lean` as the function `Proc.substNames (R T : FPNName) : Proc → Proc`, replacing all explicit free occurrences of the target name T with the replacement name R . Since bound channels are represented by de Bruijn indices, this operation acts only on explicit free channel leaves and does not require α -renaming in order to pass under channel binders, since a substituted free name cannot be captured by a binder represented as an index. Name substitution alone does not, however, ensure that the replacement name is fresh for the remaining free names or compatible with the linear resource structure of a typing judgement. Such conditions must be established separately when substitution is used in typing or metatheoretic arguments.

At the leaves, `Channel.subst` replaces a matching free channel and leaves bound indices unchanged, while dependency sets are handled pointwise by `Finset.subst`:

```
def Channel.subst (R T : FPNName) : Channel → Channel
| .free x => if x == T then .free R else .free x
| .bound i => .bound i

def Finset.subst (zs : Finset Channel) (R T : FPNName) : Finset Channel :=
  zs.image (fun u => u.subst R T)
```

Type substitution is defined in `Processes/SubstTypes.lean` as `Proc.substTypes (A : Types) (k : Nat) : Proc → Proc`, substituting A for the de Bruijn type index k throughout a process. Whenever the traversal encounters an embedded type, such as the type argument of an output prefix, it delegates to the type-level substitution operation `Types.subst`. When crossing an input prefix, which binds a type variable, the target depth is incremented and the substituted type is shifted, mirroring the binder case of `Types.subst` from [Section 4.1.1](#):

```
def Proc.substTypes (A : Types) (k : Nat) : Proc → Proc
| .input x P => .input x (P.substTypes (A.shift 0 1) (k + 1))
```

A consequence of this representation is that no analogous channel shifting operation is required. In a pure de Bruijn encoding, variables are represented as relative indices and must be shifted when crossing binders. Free channels, however, are represented as explicit identifiers rather than relative positions, so their representation is unaffected by binder depth. This avoids structural adjustment of channel names during substitution and simplifies the process-level infrastructure.

Type shifting. `Processes/SubstTypes.lean` also defines `Proc.shiftTypes (d c : Nat) : Proc → Proc`, which lifts the type-level shifting operation to processes. The traversal is uniform across most constructors, delegating to `Types.shift` whenever an embedded type is encountered, such as the type argument of an output prefix. The only non-trivial case is input, which binds a type variable and therefore increments the cutoff depth d before traversing its continuation, mirroring the binder case of `Types.shift`:

```
def Proc.shiftTypes (d c : Nat) : Proc → Proc
| .output x P A => .output x (P.shiftTypes d c) (A.shift d c)
| .input x P      => .input x (P.shiftTypes (d + 1) c)
| ...
```

Structural properties. `Processes/Lemmas.lean` establishes the fundamental structural properties required by the later typing and operational metatheory. The lemmas fall into three groups.

The first group supports the typing invariants. The lemmas `Proc.f_open_erase` and `Proc.f_open_two_erase` establish that opening a process with fresh names and then erasing those names from its free-name set recovers the original free names. They are used in the proof of `Typing_f_eq_names`. The lemmas `Proc.lc_of_open_gen` and `Proc.lc_of_open_two` show that opening a locally closed abstraction with locally closed names yields a locally closed process. They therefore support the proof of `Typing_preserves_lc_proc`, which establishes that well-typed processes are locally closed.

The second group supports the substitution lemmas. The lemmas `Proc.open_substNames_comm` and `Proc.openCut_substNames_comm` establish that opening commutes with name substitution. Their counterparts for type substitution, `Proc.open_substTypes_comm` and `Proc.openCut_substTypes_comm`, establish the analogous property for substitution of types under binders. These results are used in the typability proof, where co-finite witnesses from the typing rules must be reconciled with the concrete names introduced by process transitions.

The final group concerns freshness under opening. The lemma `Proc.not_mem_f_open` shows that opening introduces only the selected name. Thus, a different name that is absent from the original process remains absent from the opened continuation. This fact is used later when transitions and typing proofs reason about opened binder bodies under freshness side conditions.

Notation. `Processes/Notation.lean` introduces process prefix notation mirroring the locally nameless syntax established in [Section 3.2](#). For example, $\#x[[\bullet]]P$ maps to `Proc.tensor (#x) P` and $\nu((\bullet, \bullet))P$ maps to `Proc.cut P`. Parallel composition and the terminated process are written $P \mid_p Q$ and 0 respectively. The full notation is listed in [Listing 5](#).

4.1.3 Environments. Environments are defined in `Environment/Basic.lean` as `Env`, an abbreviation for a list of channel-type pairs:

```
abbrev Elem := (FName × Types)
abbrev Env  := List Elem
```

The choice of `List` as the underlying representation deserves explanation. Theoretical environments are unordered and duplicate-free, suggesting `Finset` as a natural implementation. However, since `Finset` is implemented as a quotient over lists, it complicates structural induction, pattern matching, and inversion over environments. This proved inconvenient in a development where typing derivations are frequently inverted and typing environments must be rearranged explicitly. A second attempt using association lists (`AList`) encountered similar proof-engineering difficulties. The final design therefore uses plain lists and recovers the unordered structure through an explicit permutation relation, trading quotient reasoning for structural induction and simpler interaction with Lean’s tactic machinery.

Recovering the PCM structure. Environment composition is implemented as list concatenation, yielding a simple total merge operation:

```
abbrev Env.merge (Γ Δ : Env) : Env := Γ ++ Δ
infixl:69 ", " => Env.merge
```

In the formalisation, environment composition is represented by list concatenation. Because Lean reserves the ordinary comma for syntactic purposes, the concrete notation uses the comma-like symbol “`,`”, as in $[\Gamma, \Delta]$. However, the ordinary comma is what is used to render this operation throughout the thesis.

This representation does not make environments a partial commutative monoid by definition. List concatenation is total and order-sensitive, meaning that associativity and identity are inherited from concatenation, while commutativity must be recovered up to permutation. The formalisation therefore uses Lean's built-in `List.Perm` relation, exposed through the `HasPerm` interface and written $\Gamma \sim \Delta$. The corresponding monoidal laws include:

```
lemma Env.merge_unitL (Γ : Env) : ∅, Γ = Γ
lemma Env.merge_unitR (Γ : Env) : Γ, ∅ = Γ
lemma Env.merge_assoc (Γ Δ Ξ : Env) : Γ, Δ, Ξ = Γ, (Δ, Ξ)
lemma Env.merge_comm (Γ Δ : Env) : Γ, Δ ~ Δ, Γ
```

The partiality condition is enforced separately by predicates on the merged resources. The function `Env.names` extracts the domain of an environment as a finite set of channel names. Mutual compatibility is expressed by disjointness of these domains, while internal linearity is expressed by the absence of duplicate names:

```
def Env.names (Γ : Env) : Finset FPNome := (Γ.map Prod.fst).toFinset
def Env.disjoint (Γ Δ : Env) : Prop := Disjoint Γ.names Δ.names
def Env.Nodup (Γ : Env) : Prop := (Γ.map Prod.fst).Nodup
```

The connection between these predicates and merge is captured by `Env.Nodup_merge_iff`, which states that a merged environment is linear exactly when both component environments are individually linear and their domains are disjoint:

```
lemma Env.Nodup_merge_iff {Γ Δ : Env} :
  (Γ, Δ).Nodup ↔ (Γ.Nodup ∧ Δ.Nodup ∧ Γ.disjoint Δ) := by ...
```

The proof reduces directly to the corresponding property of list append, `List.nodup_append`, together with the finite-set formulation of environment-domain disjointness.

Server usability. The typing rule for server replication requires all dependency channels to carry exponential request types. This side condition is captured by `Env.serverUsable`, which asserts that every type in an environment satisfies `Types.isServerUsable`, i.e. has the form $?B$.

```
def Env.serverUsable (Γ : Env) : Prop :=
  ∀ p, p ∈ Γ → p.snd.isServerUsable
prefix:max "?_e" => Env.serverUsable
```

Lifted operations. Since environments contain types, the locally nameless operations from [Section 4.1.1](#) are lifted pointwise over the type component of each `Elem`. Local closure, type shifting, name substitution, and type substitution are all defined by mapping over the list. The file `Environment/Lemmas.lean` proves that these operations preserve the names, disjointness, nodup, and permutation properties of environments.

The interaction between shifting and local closure for environments is captured by `Env.lc_shift_inv`. When an environment is moved under a newly introduced type binder, shifting adjusts its bound type indices so that its scoping structure is preserved at the increased binder depth. Formally, the lemma states that $(\Gamma^+).lc(n+k+1) \iff \Gamma.lc(n+k)$. Thus, local closure of the shifted environment at the increased depth corresponds precisely to local closure of the original environment at the preceding depth. A specialisation of this result is used later to show that typing judgements contain only locally closed syntax.

4.1.4 Hyperenvironments. Hyperenvironments are defined in `HyperEnvironment/Basic.lean` as `HyperEnv`, an abbreviation for a list of environments:

```
abbrev HyperEnv := List Env
abbrev HyperEnv.merge (G H : HyperEnv) : HyperEnv := G ++ H
infixl:55 " |h " => HyperEnv.merge
```

As for environments, merging is implemented as list concatenation, using $\emptyset$ as the unit and $G|_h H$ as notation for composition. Associativity and unit laws therefore follow from the underlying list structure. However, recovering the commutative structure requires more than the ordinary list permutation relation, since hyperenvironments are lists whose elements are themselves environments considered modulo permutation.

Deep permutation. For environments, Lean’s `List.Perm` relation is sufficient, since it allows arbitrary reordering of channel-type pairs. For hyperenvironments, however, permutation must operate at two levels: it must allow reordering of component environments, and it must also respect permutation equivalence inside each component environment.

For example, consider the hyperenvironment containing a single component environment $[x : A, y : B]$. Ordinary `List.Perm` treats this component environment as an opaque element, so it cannot identify it with $[y : B, x : A]$, which would be the desired behaviour. Thus, a custom relation `HyperEnv.Perm`, which permits component environments to be replaced by permutation-equivalent environments, was developed.

```
inductive HyperEnv.Perm : HyperEnv → HyperEnv → Prop where
| nil : Perm [] []
| cons : {Γ Δ : Env} {G H : HyperEnv} : Γ ~ Δ → Perm G H → Perm (Γ :: G) (Δ :: H)
| swap : {Γ Δ : Env} {G : HyperEnv} : Perm (Γ :: Δ :: G) (Δ :: Γ :: G)
| trans {G H J : HyperEnv} : Perm G H → Perm H J → Perm G J
```

The key constructor is `cons`. Unlike ordinary list permutation, it does not require the head environments to be syntactically identical, it only requires them to be permutation-equivalent. Thus `HyperEnv.Perm` is a deep permutation relation, propagating equivalence both across the ordering of component environments and within the environments themselves.

Well-formedness. Hyperenvironment well-formedness is decomposed into two predicates. The predicate `HyperEnv.Nodup` requires each component environment to be internally linear, while `HyperEnv.PairwiseDisjoint` requires distinct component environments to have disjoint domains:

```
def HyperEnv.Nodup (G : HyperEnv) : Prop :=
  ∀ Γ, Γ ∈ G → Env.Nodup Γ

def HyperEnv.PairwiseDisjoint (G : HyperEnv) : Prop :=
  List.Pairwise Env.disjoint G
```

The distinction is important because the two predicates rule out different violations of linearity. For example, assuming x , y , and z are distinct names:

- $[x : A] |_h [y : B, z : C]$ satisfies both predicates.
- $[x : A, x : B] |_h [y : C]$ violates `Nodup` (x appears twice within one component).
- $[x : A] |_h [x : B, y : C]$ violates `PairwiseDisjoint` (x appears within distinct components).

Together, these predicates enforce global linearity in that every channel appears at most once across the whole hyperenvironment.

Lifted operations. Type shifting, name substitution, and type substitution are lifted pointwise from environments to hyperenvironments. The accompanying lemmas in `HyperEnvironment/Lemmas/Basic.lean` establish how these operations interact with the structural properties required later, including names, disjointness, pairwise disjointness, and deep permutation. Since the definitions follow uniformly from their environment-level counterparts, they are omitted here and included in the accompanying Lean archive.

4.1.5 Typing judgements. The typing relation is defined in `Judgement/Basic.lean` as the inductive predicate `Typing`:

```
inductive Typing : Nat → Proc → HyperEnv → Prop
```

We write $n \vdash P :: \mathcal{G}$ for the judgement that P is typed under the hyperenvironment $\mathcal{G}$ at type-binder depth n . An inhabitant of this proposition is a typing derivation. The index n tracks the current type-binder depth, while the remaining indices record the process and hyperenvironment appearing in the judgement. These indices are later exposed by the projection functions used to relate typed transitions to the process-level and environment-level semantics. The full inductive definition, including the typing constructors and exchange rules, is given in [Listing 9](#).

The process syntax and typing derivations occupy distinct levels of the formalisation. Terms of type `Proc` represent raw process syntax and may be constructed independently of whether they admit a typing derivation. By contrast, a term inhabiting $n \vdash P :: \mathcal{G}$ is a derivation that P uses the resources in $\mathcal{G}$ according to the typing rules: each constructor requires the typed premises and structural side conditions needed for its conclusion. Properties such as local closure, linearity, and correspondence between free process names and hyperenvironment names are therefore established as invariants of valid typing derivations rather than imposed on the raw syntax datatype.

Type-binder depth. The type-binder depth is relevant whenever the derivation crosses a polymorphic binder. For example, the `forall_` constructor increments the depth from n to $n + 1$ and shifts the surrounding environment to keep embedded type indices in scope:

```
| forall_ :  
  Typing (n + 1) P [x:B :: Γ+] →  
  Typing n (#x(◇)).P [x:∀.B :: Γ]
```

The shift Γ^+ abbreviates shifting the type indices in Γ by one starting at depth 0, mirroring the binder-sensitive shifting used by type substitution in [Section 4.1.1](#).

The type-binder depth is also relevant for the `mix` rule, which requires both sub-derivations to share the same depth n . When derivations are available at different depths, they can be brought into alignment using `Typing_weakening`. This lemma states that a derivation at depth n can be lifted to depth $n + c$ by shifting the embedded type indices in both the process and the hyperenvironment:

```
lemma Typing_weakening :  
  Typing n P  $\mathcal{G} \rightarrow \forall d\ c, \text{Typing } (n + c) (P \uparrow d, c) (\mathcal{G} \uparrow d, c) := \text{by ...}$ 
```

This provides the weakening principle needed to compose derivations across polymorphic binders while keeping embedded type indices in scope.

Exchange and explicit side conditions. In some constructors, the formalised rule chooses an ordering different from the paper presentation so that the entry manipulated by the rule occurs at the head of the list representing its environment component. This is an implementation-oriented choice and does not affect the typing discipline. The order-insensitive interpretation of environments and

hyperenvironments is recovered by exchange rules in the typing relation, which permit reasoning up to permutation:

```
| exchange_env   : Typing n P (Γ :: G) → Γ ~ Δ → Typing n P (Δ :: G)
| exchange_hyper : Typing n P G → G ~ H → Typing n P H
```

The exchange rules recover the unordered character of the paper-based presentation without quotienting environments or hyperenvironments. The complementary issue is partiality. This stems from merging being implemented as total list concatenation. Thus, typing rules have been formalised to carry explicit disjointness and freshness premises to ensure that only valid linear compositions are formed. For example, `mix` requires the two hyperenvironments to be disjoint, while prefix rules such as `tensor` and `parr` require freshness premises ensuring that subject channels do not already appear in the continuation environment.

The same design principle reappears later in the operational semantics, where transitions must also respect permutation equivalence of environments and hyperenvironments.

Co-finite quantification. Binding rules use co-finite quantification as described in [Section 3.4](#). For example, the `parr` constructor types an input prefix by requiring the opened continuation to be well typed for every fresh channel name outside a finite set L :

```
| parr (hF : x ∉ Γ.names) (L : Finset FPNName) :
  (∀ y ∉ L, Typing n (P((#y))) [y:A :: x:B :: Γ]) →
  Typing n (x((•)).P) [x:A ⋈ B :: Γ]
```

Here, the process carries no explicit bound channel name. Instead, the bound occurrence in the continuation is instantiated by opening P with a fresh name y . The premise `hF` enforces that the subject channel x is not already present in the continuation environment. Rules binding multiple names, such as `cut`, follow the same pattern but quantify over two distinct fresh names.

Structural invariants. `Judgement/Names.lean` establishes the central name invariant for the typing relation, named `Typing_f_eq_names`. It states that, for any well-typed process, the free names of the process coincide exactly with the domain of its hyperenvironment.

$$n \vdash P :: G \implies P.f = G.names$$

The proof proceeds by induction on the typing derivation. Most cases follow by unfolding `Proc.f` and `HyperEnv.names` and rewriting with the induction hypothesis. In the co-finite binding cases, the proof instantiates fresh names using `exists_two_fresh`, applies the induction hypothesis to the opened process, and then uses `Proc.f_open_erase` to cancel the introduced names from the free name set and recover equality for the closed process.

This ensures exact correspondence between process free names and typed environment entries: no free process channel is left untyped, and no environment entry is unused.

Building on this, `Judgement/Properties/Linearity.lean` proves that any well-typed hyperenvironment is pairwise disjoint and all component environments have no duplicate names.

$$n \vdash P :: G \implies \text{Nodup } G \wedge G.\text{PairwiseDisjoint}$$

The proof proceeds by induction on the typing derivation. Most non-binding rules are discharged uniformly by unfolding the relevant definitions and applying the induction hypothesis. The co-finite binding cases (`cut`, `tensor`, `parr`, and contraction (`c`)) require instantiating fresh names and reconstructing both nodupness and disjointness of the component environments from the opened induction hypothesis. The exchange cases use preservation of these properties under permutation.

Together, `Typing_f_eq_names` and `Typing_preserves_linearity` guarantee that typing derivations can only be constructed for well-formed, linear contexts. In particular, a well-typed process cannot contain duplicate free channel occurrences: by the name correspondence, duplicate free names would require duplicate entries in the hyperenvironment, contradicting the linearity constraints.

A third invariant, `Typing_preserves_lc`, establishes that every well-typed process is locally closed and that the types occurring in each component of its hyperenvironment are locally closed at the current type-binder depth. The proof is split into `Typing_preserves_lc_proc` for the process and `Typing_preserves_lc_context` for the hyperenvironment, and the two results are then packaged together. Both proofs proceed by induction on the typing derivation. In the co-finite binding cases, the typing premise provides local closure only for continuations opened with sufficiently fresh names. The lemmas `Proc_lc_of_open_gen` and `Proc_lc_of_open_two` recover local closure of the corresponding binder-containing process. The polymorphic cases additionally require inversion lemmas for the transformations appearing in their premises. The lemma `Env_lc_shift_inv_0`, derived pointwise from `Types_lc_shift_inv_0`, recovers local closure of an environment before it is shifted beneath a type binder. Given local closure of the substituted type, `Types_lc_subst_inv_0` recovers local closure of a type body at depth $n + 1$ from local closure of its instantiated form at depth n .

Substitution. `Judgement/Subst.lean` establishes that typing is stable under both name substitution and type substitution. The theorem `Typing_substNames` states that, from a derivation $n \vdash P :: \mathcal{G}$, substituting a free name y for a free name x throughout both the process and its hyperenvironment yields the corresponding derivation $n \vdash P\{y//x\} :: \mathcal{G}\{y//x\}$. The required side condition mandates that if there is a pre-existing occurrence of y in the hyperenvironment then $y = x$, in which case the substitution is the identity operation. Consequently, when x and y are distinct, y must be fresh for the hyperenvironment, preventing two distinct linear resources from being assigned the same channel name after substitution.

The theorem `Typing_substTypes` establishes the corresponding result for type substitution. Given a derivation $n + 1 \vdash P :: \mathcal{G}$, a type A that is locally closed at depth n , and an admissible substitution index $k \leq n$, substituting A for index k throughout the process and hyperenvironment yields a derivation $n \vdash P\{A//k\} :: \mathcal{G}\{A//k\}$. Thus, type substitution reduces the type-binder level by one while preserving typability. Both substitution proofs proceed by induction on the typing derivation and rely on the corresponding commutation properties of opening and substitution for processes and types.

4.2 Semantics

The dynamic behaviour of the π MLL fragment is formalised in the `Semantics/namespace`. Following the erasure structure described in [Section 2.5](#), the development contains three related transition systems: a typed transition system over typing derivations, and two projected systems describing the induced process and environment behaviour.

- **TypingStep_m**: the primary, resource-aware LTS over typing derivations (`JudgementStep/Basic.lean`);
- **ProcStep_m**: the process-level LTS obtained by erasing typing resources (`ProcStep/Basic.lean`);
- **EnvStep_m**: the resource-level LTS describing the corresponding evolution of hyperenvironments (`EnvStep/Basic.lean`).

This three-level organisation directly reflects the semantic architecture of [Montesi and Peressotti \[2021\]](#): the paper takes transitions of typing derivations and obtains process-level and environment-level behaviour through erasure. The mechanisation preserves this organisation, representing each level as an inductive relation. The main presentational difference is therefore the formulation of their correspondence. Rather than expressing erasure through equality of labelled successor sets, as in the paper presentation, the Lean development formulates the correspondence through relational projection and lifting lemmas. The complete inductive definitions of the three transition systems are given in [Listings 12 to 14](#).

4.2.1 Labels. Transition labels are formalised in `Labels.lean`, directly realising the label algebra introduced in [Section 2.5](#). Although the operational semantics mechanised here target π MLL, the action vocabulary is defined for the full π LL calculus to support future extensions. The free and introduced name functions “f” and “i” are realised as `Lbl.f` and `Lbl.i`, computed by structural recursion on the label and used in the side conditions of the [rules `PAR1`](#), [SYN](#) and [PAR₂](#). The disjointness condition on par labels is enforced as the decidable predicate `Lbl.WF`, which appears as an explicit side condition in the parallel synchronisation rules.

The core action type, `Act`, captures observable communication events, including empty communication, channel transmission, type instantiation, and server interaction. Selection and server actions are parameterised by the auxiliary datatype `Mu`. The `Lbl` inductive then extends observable actions with the silent transition τ , explicit session-link labels, and a parallel constructor combining two observable actions directly, reflecting the synchronised action labels used by the parallel rules.

```

inductive Act : Type
| one      (x : FPNName)
| tensor   (x y : FPNName)
| output   (x : FPNName) (A : Types)
| muBrack  (x : FPNName) ( $\mu$  : Mu)
| ...

inductive Lbl : Type
| tau
| link (x y : FPNName)
| act  (p : Act)
| par  (l l' : Act)

```

The label notation is overloaded through the `HasBracket` and `HasParen` typeclasses. Since labels record concrete observable actions, their subjects are free process names rather than locally nameless channels. The overloading is therefore confined to the label language. Bracketed and parenthesised labels are distinguished by the type of their arguments, allowing empty communication, channel transmission, type communication, and server actions to share a compact notation. The corresponding notation instances are given in [Listing 6](#).

4.2.2 The typed transition system. The primary LTS, `TypingStepm`, is a relation between typing derivations. Its constructors mirror the operational rules shown in [Figure 2](#), but make every transition intrinsically typed by relating a source typing derivation to a target typing derivation.

```

inductive TypingStepm :
  Typing n P  $\mathcal{G}$   $\rightarrow$  Lbl  $\rightarrow$  Typing n' P'  $\mathcal{G}' \rightarrow$  Prop

```

The corresponding process and environment behaviour is captured by the projected relations `ProcStepm` and `EnvStepm`, which are used in the subsequent session-fidelity development.

As established previously, both environment and hyperenvironment composition are formalised using total list appends. Consequently, the explicit disjointness proofs required by the static Typing relation to safely mix environments must propagate directly into the dynamic TypingStep_m relation. For instance, the par₁ constructor requires explicit hypotheses (hD1 and hD2) to guarantee that the hyperenvironments remain strictly disjoint both before and after the transition:

```
| par1
  (hD1 :  $\mathcal{G}.\text{disjoint } \mathcal{H}$ ) (hD2 :  $\mathcal{G}'.\text{disjoint } \mathcal{H}$ ) (disj :  $l.i \cap Q.f = \emptyset$ )
  (h : TypingStepm  $\mathcal{D} \ 1 \ \mathcal{D}'$ ) :
  TypingStepm
    (Typing.mix hD1  $\mathcal{D} \ \mathcal{E}$ ) 1 (Typing.mix hD2  $\mathcal{D}' \ \mathcal{E}$ )
```

Listing 1. The par₁ constructor of the typed transition relation, with explicit pre- and post-transition disjointness premises.

The post-state disjointness proof hD2 is included explicitly as a proof-engineering convenience. It is recoverable from the pre-state disjointness proof hD1, the freshness condition $i(l) \cap Q.f = \emptyset$, the name correspondence invariant Typing_f_eq_names, and TypingStep_m_names_bound, which states that the target hyperenvironment contains no names beyond those already present in the source or introduced by the label. Keeping hD2 as a premise avoids reconstructing this derived disjointness fact repeatedly in later proofs, at the cost of making the transition relation slightly less minimal. The signatures for the lemmas describing transition name bounds are stated in Listing 21.

Strictly stepping typing derivations can expose structural artefacts that are usually hidden in the paper presentation. For example, in Figure 3, the intermediate derivation is typed by $\emptyset \mid [y:1]$ rather than simply $y:1$, because the MIX rule preserves the empty component contributed by the terminated left process. In Lean, this corresponds to a Typing.mix constructor containing a Typing.mix₀ sub-derivation. The result is valid, but reducing it to the canonical form requires applying the monoidal identity laws of the hyperenvironment explicitly, rather than by definitional equality. The final $\equiv$ step in the figure, which additionally uses process congruence to identify $0 \mid_p 0$ with 0 , is not currently automated because structural congruence has not been mechanised. This limitation is discussed further in Section 6.

4.2.3 Projected transition systems. The process and environment transition systems are defined to capture the two erasure views of a typed transition. ProcStep_m records the process behaviour obtained by forgetting typing resources, while EnvStep_m records the corresponding resource evolution. These relations are later connected to TypingStep_m by the simulation theorems in Section 5.1.

Process steps. ProcStep_m models the syntactic evolution of processes independently of typing resources. In rules involving binders, the transition label exposes the concrete name selected by the action, so the continuation is opened with that name and the required freshness conditions are recorded directly as premises. For example, the tensor rule opens the continuation with the name y transmitted by the label, requiring y to be fresh for both the subject channel and the continuation:

```
| tensor (hF :  $y \notin \{x\} \cup P.f$ ) :
  ProcStepm ( $\#x \ll \bullet \cdot P$ ) ( $x \ll y \cdot$ )  $P((\#y))$ 
```

This differs from the co-finite formulation of the typing rule. The process transition records the concrete name chosen by an operational step, whereas the typing rule provides a derivation for every name outside a finite exclusion set. The typability proof in Section 5.1 reconciles these views by instantiating the co-finite premise with the name exposed by the process transition.

Environment steps. EnvStep_m formalises transitions of typing hyperenvironments, written $\mathcal{G} \xrightarrow{l} \mathcal{H}$. Its prefix rules record the protocol-level effect of observable actions. For example, a free channel typed by 1 is consumed by an empty output action, while a free channel typed by $\perp$ is consumed by an empty input action. The relation is closed under hyperenvironment composition and permutation, reflecting the list-based representation of environments and hyperenvironments.

The synchronisation and restriction constructors capture internal communication between dual channels. In particular, once the communicating channels are hidden by restriction, their synchronisation produces a τ -transition on the surrounding hyperenvironment without exposing the internal resource transformation. The subsequent metatheory relates this environment-level behaviour to the intrinsically typed transitions of TypingStep_m .

5 Metatheory

This section describes the metatheoretic results for the typed, process, and environment transition systems defined in [Section 4.2](#). The development is contained in the `SessionFidelity/` namespace and concerns the mechanised π MLL fragment.

5.1 Session fidelity

Session fidelity states that well-typed processes evolve according to their session types and that their associated linear resources evolve consistently with each process transition. The mechanisation establishes the two projection results from typed transitions to process and environment transitions. The converse lifting direction is developed through typability subject reduction, with its remaining proof obligation isolated to the `res` case. Completing this case would finalise the typability-preservation result for the multiplicative fragment.

Erasure simulations. As shown by [Montesi and Peressotti \[2021, Theorem 4.7\]](#), erasure induces a strong bisimulation between lts_p and lts_d . The mechanisation currently establishes the projections from typed derivation steps to their process and environment behaviours. The files `ProcStep.lean` and `EnvStep.lean` contain the corresponding theorems: `session_fidelity_procm` projects a typed transition to a process transition, while `session_fidelity_envm` projects it to an environment transition. The former corresponds to the forward direction of the erasure result of [Montesi and Peressotti \[2021\]](#), while the latter corresponds to their derivation-level session-fidelity property.

```
theorem session_fidelity_procm
  {D : Typing n P G} {D' : Typing n' P' G'}
  (hStep : TypingStepm D 1 D') :
  ProcStepm (proc D) 1 (proc D')
```

```
theorem session_fidelity_envm
  {D : Typing n P G} {D' : Typing n' P' G'}
  (hStep : TypingStepm D 1 D') :
  EnvStepm (env D) 1 (env D')
```

Both projection proofs proceed by induction on the typed transition derivation, and most constructors map directly to their process-level or environment-level counterpart. The main additional work occurs in the name-binding prefix cases `tensor` and `parr`. For the process projection, the free-name side conditions required by `ProcStepm` are reconstructed by selecting an auxiliary fresh name, applying the corresponding all-fresh typing lemma, and combining `Typing_f_eq_names`, `Typing_preserves_linearity`, and the fact that opening preserves existing free-name occurrences. For the environment projection, the same cases require only the all-fresh typing lemma and

`Typing_preserves_linearity`, since the target relation records the evolution of typing resources rather than freshness of an opened process continuation. The remaining constructors reuse the structural and freshness information carried by the typed transition. The Lean statements of the resulting process- and environment-projection theorems are given in [Listings 15](#) and [16](#).

Typability. The target lifting theorem is formulated in `Typability.lean`. It corresponds to the reverse direction of the erasure correspondence: given a process transition from a well-typed process, the transition should lift to the level of typing derivations. Concretely, if $n \vdash P :: \mathcal{G}$ and $P \xrightarrow{l} P'$, then there should exist a hyperenvironment $\mathcal{G}'$ and a typing derivation $n \vdash P' :: \mathcal{G}'$ connected to the source derivation by a typed transition labelled l . The proof is complete for the multiplicative cases except `res`, whose remaining obligation is discussed in [Section 6](#).

In the paper presentation, erasure is expressed through a homomorphism condition, which can be read extensionally as stating that, for each label l , the process successors reachable from $\text{proc}(\mathcal{D})$ by an l -labelled transition are exactly the projections of the derivation successors reachable from $\mathcal{D}$ by the same label. In the Lean formalisation, transitions are represented as inductive relations, and this correspondence is instead formulated through projection and lifting lemmas. The theorem `session_fidelity_proc_m` establishes the projection from typed derivation steps to process steps, while `typability_subject_reduction_m` develops the converse lifting direction for transitions of well-typed processes. This relational formulation avoids introducing explicit sets of successors and allows the proofs to proceed by induction on the relevant transition derivations.

```
theorem typability_subject_reduction_m
  ( $\mathcal{D} : \text{Typing } n \text{ } P \mathcal{G}$ ) ( $\text{hPS} : \text{ProcStep}_m \text{ } P \text{ } P'$ ) :
   $\exists (\mathcal{G}' : \text{HyperEnv}) (\mathcal{D}' : \text{Typing } n \text{ } P' \mathcal{G}'),$ 
   $\text{TypingStep}_m \mathcal{D} \text{ } l \mathcal{D}'$ 
```

This theorem forms the technical core of the converse lifting direction. Its proof proceeds by induction on the process transition derivation. Each case inverts the source typing derivation to recover the typing rule compatible with the observed process shape, then constructs a target hyperenvironment and typing derivation for the resulting process.

The proof has two main technical obligations. First, it must reconcile the concrete freshness witnesses used by `ProcStepm` with the co-finite premises of the typing rules. For example, a process transition fixes a concrete name introduced by a prefix, while the corresponding typing rule provides derivations uniformly for all names outside some finite set. Auxiliary *all fresh* lemmas bridge this gap by showing that the co-finite premise can be instantiated with the concrete name appearing in the process step. Second, in the `one_bot`, `tensor_parr` and `res` cases, the proof must rearrange hyperenvironments to expose the components containing the communicating endpoints. In particular, transition under restriction requires opening the restricted process with fresh endpoints, applying the induction hypothesis to the opened body, and reconstructing the target hyperenvironment up to permutation. This reconstruction remains unsolved for the `res` case.

The res case. The remaining incomplete case is `res`. Inverting the source typing derivation exposes the body of the restriction under two fresh dual endpoints, yielding a co-finite premise of the form:

$$\forall x \notin L, \forall y \notin L, x \neq y \rightarrow n \vdash Q((x, y)) :: \mathcal{H} \mid_h [x : A : : \Gamma] \mid_h [y : A^\perp : : \Delta]$$

The process transition already supplies concrete fresh representatives for these bound endpoints. Using the freshness premises of the process step together with the name-correspondence invariant `Typing_f_eq_names`, the current proof establishes that these representatives are also fresh for the

hyperenvironment components exposed by inversion. The lemma `Typing_res_all_fresh` can then instantiate the co-finite typing premise at the concrete names x and y , allowing the induction hypothesis to be applied to the opened body.

The unresolved difficulty arises after this application of the induction hypothesis, as it returns a typed transition for the opened target body under an existentially determined generic target hyperenvironment. To reconstruct a typing derivation for the closed target process, this hyperenvironment must be reorganised into a form that exposes the two dual endpoint resources required by the enclosing cut constructor. In particular, the proof needs sufficient structural information to recover components containing $x : A$ and $y : A^\perp$ in the post-transition hyperenvironment, or to account for the case in which the transition has merged the relevant resources into a single component.

The established name-preservation results are insufficient for this reconstruction. The lemma `TypingStep_m_names_preserved` records that names unaffected by the transition label remain present after the transition, but it does not identify the environment components in which those names occur. Membership of x and y in the post-transition name set therefore does not by itself imply that the target hyperenvironment can be permuted into the form:

$$\mathcal{H}' \mid_h [x : A :: \Gamma'] \mid_h [y : A^\perp :: \Delta']$$

The current proof attempt therefore introduces stronger auxiliary invariants for typed transitions, stated in [Listing 22](#). These invariants progressively strengthen component preservation from transporting an unaffected hyperenvironment component, to tracking a distinguished endpoint-bearing component, to recovering the two dual components required under restriction. The final invariant, `TypingStep_m_preserves_disjoint_or_merges`, is intended to supply the post-transition hyperenvironment structure needed in the `res` case of `typability_subject_reduction_m` to reconstruct the enclosing cut derivation. The remaining work concerns the nested restriction arguments required to establish this stronger component-level preservation result. This exposes a proof-engineering consequence of representing hyperenvironments as lists modulo permutation, since reasoning about the persistence and placement of particular typed resources requires dedicated reconstruction lemmas.

Subject reduction. The subject-reduction statement is formulated in `SubjectReduction.lean`. It states that, if a well-typed process takes a process transition, then the target process is well typed under some evolved hyperenvironment, and that this hyperenvironment evolves under the same label. Once `typability_subject_reduction_m` is completed, the result follows directly by applying the environment simulation `session_fidelity_env_m` to the lifted typed transition.

```
theorem subject_reduction_m
  ( $\mathcal{D}$  : Typing n P  $\mathcal{G}$ ) (hPS : ProcStep_m P 1 P') :
   $\exists \mathcal{G}', \text{EnvStep}_m \mathcal{G} 1 \mathcal{G}' \wedge (\text{Typing n P' } \mathcal{G}') := \text{by}$ 
  obtain  $\langle \mathcal{G}', \mathcal{D}', \text{hTS} \rangle := \text{typability\_subject\_reduction}_m \mathcal{D} \text{ hPS}$ 
  have hES := session_fidelity_env_m hTS
  exact  $\langle \mathcal{G}', \text{hES}, \mathcal{D}' \rangle$ 
```

The proof is a direct composition of the previous results: `typability_subject_reduction_m` lifts the process step to a typed derivation step, and `session_fidelity_env_m` projects that typed step to the corresponding environment transition. This realises the process-level session fidelity result corresponding to Corollary 4.10 of [Montesi and Peressotti \[2021\]](#).

Interpretation. The mechanised metatheory therefore establishes both projections from typed derivation transitions and completes the reverse lifting argument for all multiplicative cases except

res. Completion of that remaining case would yield process-level session fidelity for the mechanised π MLL fragment by composing the lifting result with the environment projection.

The paper additionally establishes broader behavioural and concurrency results, including similarity and bisimilarity, the diamond property, serialisation, non-interference, readiness, and productivity [Montesi and Peressotti 2021]. These results are not mechanised in the present development and are discussed as directions for future work in Section 7.

6 Discussion

This section reflects on the design choices made throughout the formalisation, discusses alternatives explored during development, and identifies limitations of the resulting Lean development.

6.1 Proof architecture

The architecture of the transition semantics was revised during development. An earlier formulation did not introduce a transition system over typing derivations and instead attempted to relate process transitions directly to environment transitions. Under this approach, the typability proof relied on inversion of environment transitions to recover the hyperenvironment structure needed to reconstruct a typing derivation after a process step. Consequently, the environment transition relation accumulated additional freshness and structural premises whose purpose was to support the metatheoretic proof rather than to describe the resource-level transition itself.

This approach was abandoned because it obscured the intended role of the environment semantics and diverged from the derivation-level organisation of the paper presentation. In the current formulation, EnvStep_m contains only the premises required to describe valid environment-level transitions. The stronger freshness, linearity, and component-preservation information required for reconstruction is instead carried by TypingStep_m and its auxiliary invariants. The theorem $\text{session_fidelity_env}_m$ then establishes the forward projection from typed transitions to environment transitions.

6.2 Proof-engineering trade-offs

Several premises in the typing and transition relations are included for proof convenience rather than logical necessity. The most prominent example is the post-state disjointness proof hd2 in the parallel constructors par_1 and par_2 of TypingStep_m . The former is shown in Listing 1. This premise is derivable from the pre-state disjointness proof hd1 , the freshness condition requiring the names introduced by the label to be disjoint from the non-stepping component, and the structural property that transitions introduce only names recorded by their labels. Keeping hd2 explicit avoids reconstructing this derived fact in every inductive case, at the cost of slightly cluttering the inductive definition. This is representative of a broader proof-engineering trade-off: some redundancy in the definitions can make individual proofs shorter and more robust, even when it makes the relations less minimal.

A second source of proof overhead is the mismatch between concrete freshness witnesses in ProcStep_m and co-finite premises in the typing rules. The binding constructors of ProcStep_m record the particular names introduced by a transition, whereas the corresponding typing rules are stated co-finitely. The simulation proofs must therefore show that the co-finite typing premise can be instantiated with the concrete names chosen by the process transition. This is handled by auxiliary *all-fresh* lemmas, such as those for the tensor and parr prefix binders. Thus, the current architecture keeps the process transition relation concrete and structurally direct, at the cost of additional freshness-bridging infrastructure in the simulation proofs.

Table 4. Principal proof-engineering trade-offs in the mechanisation.

Mechanised design choice or simplification	Resulting proof obligation or trade-off
The locally nameless representation avoids explicit reasoning up to α -equivalence for bound channel names.	Processes require opening and closing operations, local closure predicates, and lemmas connecting these operations with typing and transitions.
Co-finite premises avoid fixing a particular fresh representative in typing rules that bind channel names.	Concrete names exposed by ProcStep_m must be related back to the co-finite typing premises through auxiliary <i>all-fresh</i> lemmas.
Environments and hyperenvironments are represented as lists while their logical composition is treated as unordered.	Exchange and component rearrangement require explicit List.Perm reasoning and the corresponding hyperenvironment permutation infrastructure. In particular, the remaining <i>res</i> case requires recovering the endpoint-bearing components needed to reconstruct the enclosing cut derivation.
Process and environment behaviour is related through the intrinsically typed transition system TypingStep_m .	Forward projection results are structurally direct, while the converse typability direction requires reconstructing a typed transition from an observed process step.
A derivable post-state disjointness premise is retained explicitly in the parallel constructors of TypingStep_m .	The transition relation is less minimal, but subsequent proofs can use post-transition disjointness directly rather than reconstructing it repeatedly from auxiliary invariants.

Summary of shifted proof obligations. The proof effort exposed by the mechanisation is concentrated less in transcribing the paper rules than in establishing the infrastructure required to use those rules inductively. This shift arises from deliberate choices in the representation and semantic architecture of the formalisation. The locally nameless representation avoids reasoning directly up to α -equivalence, co-finite typing rules avoid fixing particular fresh representatives, and list-based environments provide a simple account of structural composition. These choices are accompanied by proof obligations concerning opening, freshness transfer, permutation, and reconstruction. The principal trade-offs are summarised in Table 4.

These trade-offs show that mechanisation does not merely replace informal arguments with automation. Instead, it makes explicit the auxiliary infrastructure required by the chosen representation and semantic architecture. A separate limitation, considered next, is that the formalisation does not currently include the process-level structural congruence.

6.3 Structural congruence

The current formalisation does not include a structural congruence relation for processes. In the paper presentation of π LL, structural congruence identifies processes modulo the commutative, associative, and unit laws of parallel composition, together with scope manipulation for restriction.

The mechanisation recovers the typing-level counterpart of the parallel-composition laws through the monoidal laws for hyperenvironments and the exchange rules of the typing relation. However, these results do not establish corresponding equivalences between process terms. For example, while an empty hyperenvironment component can be removed from the typing structure, the formalisation does not identify $P \mid_p 0$ with P at the process level.

This omission matters because the operational semantics is defined over concrete process syntax rather than over processes modulo structural congruence. Although the transition rules propagate actions through parallel contexts, they do not identify syntactically different but structurally equivalent process terms. Consequently, intermediate processes may retain parallel units or a particular association and ordering of parallel components even when these distinctions are immaterial in the paper presentation. Moreover, the process semantics does not currently support the scope-manipulation principles for restriction supplied by structural congruence in the paper. The current development has not shown structural congruence to be necessary for the proofs established so far, but its omission limits the correspondence between the mechanised process semantics and the structural flexibility of the paper presentation.

6.4 Current limitations

The static formalisation covers the full π LL calculus, whereas the operational semantics and metatheory currently target only π MLL. Even within this fragment, session fidelity is not yet fully mechanised: the remaining incomplete case of `typability_subject_reductionm` is the `res` case, where the post-transition structure required to reconstruct the enclosing cut derivation has not yet been recovered. Thus, the present mechanisation establishes the central preservation argument for the multiplicative fragment except for the `res` case.

The formalisation also does not yet cover several further results from the paper metatheory. In particular, the behavioural theory of similarity, bisimilarity, and congruence remains unmechanised, as do the diamond property, serialisation, non-interference, readiness, and the associated productivity result. Addressing these limitations requires both completing the remaining metatheoretic proof obligations for π MLL and extending the mechanised operational and behavioural theory towards the full calculus. These extensions are considered in the following chapter.

7 Future work

The most immediate directions for future work concern completing the remaining session-fidelity proof obligation and extending the dynamic metatheory to the full π LL calculus.

7.1 Completing session fidelity for π MLL

The immediate priority for the multiplicative fragment is to close the remaining proof gap in session fidelity. Beyond this required completion step, the existing proof development suggests two possible proof-engineering improvements: reformulating selected process transitions using co-finite premises and automating recurring structural obligations concerning typing environments.

Completing the remaining proof case. The immediate remaining proof obligation is the `res` case of `typability_subject_reductionm`. As established, completing this case requires recovering the two endpoint-bearing hyperenvironment components after applying the induction hypothesis beneath a restriction. The current proof attempt isolates this requirement into two auxiliary results: a single-block preservation lemma, needed when only one distinguished component lies beneath a stepping parallel operand, and a paired-endpoint reconstruction lemma, needed to recover the two dual components consumed by the enclosing cut. The remaining work lies in completing their

permutation-sensitive cases for parallel propagation, synchronisation, internal communication, and nested restriction transitions.

Co-finite process transitions. The current ProcStep_m relation records concrete fresh names in rules that open bound continuations, whereas the corresponding typing rules use co-finite quantification. As discussed in [Section 6](#), this asymmetry introduces proof overhead in the typability argument, where the concrete names chosen by a process transition must be reconciled with the co-finite premises of the typing derivation through auxiliary *all-fresh* lemmas. Refactoring the relevant process-transition rules, including `tensor`, `parr`, `one_bot`, `tensor_parr`, and `res`, to use a co-finite formulation could align the two relations more directly and reduce this bridging infrastructure. Such a reformulation would, however, make the erased process semantics less directly concrete, and its consequences for the erasure simulation proofs would need to be investigated.

Proof automation. A recurring cost throughout the formalisation is the manual rearrangement of hyperenvironments using permutation, disjointness, name-set, and freshness lemmas. Many of these obligations amount to routine framing arguments: showing that components remain disjoint, that two hyperenvironments agree up to permutation, or that a chosen name is fresh for the relevant context. A dedicated aesop rule set, together with a curated collection of simplification lemmas for normalising name-set and disjointness goals, could automate much of this bookkeeping and significantly reduce proof size. This would not replace the main inversion and reconstruction arguments, such as the remaining `res` case, but could make those arguments substantially more manageable.

7.2 Extending to full π LL

Once the multiplicative fragment is complete, extending the formalisation to the full π LL calculus appears to require additional proof engineering rather than a new formal architecture. The static model is already defined for the full calculus, and many typing inversion lemmas for the additive, polymorphic, axiom, and exponential constructs are already available. The main work is therefore expected to lie in extending `typability_subject_reductionm` and in proving the hyperenvironment extraction lemmas needed for the additional synchronisation cases, rather than in the erasure projections or the unit-like transition axioms. Some additional work would also be needed to extend the operational semantics with the transition constructors corresponding to the full set of rules from Appendix B.2 of [Montesi and Peressotti \[2021\]](#).

The remaining prefix and unit-like transition axioms from Appendix B.2 of [Montesi and Peressotti \[2021\]](#) are expected to follow the same proof pattern as the already mechanised `one` and `bot` cases. This includes the additive selections, server-use actions, polymorphic send and receive actions, disposal and duplication send actions, and the link actions. In each case, the proof should proceed by inverting the source typing derivation, constructing the corresponding typed transition, and reusing the existing local-closure and freshness infrastructure. The polymorphic cases additionally rely on the type-substitution and shifting lemmas developed for the locally nameless representation, while the link and axiom-cut cases require the name-substitution lemmas. The axiom-cut interaction may require special treatment, since it combines restriction with name substitution; one possible approach is to add an explicit axiom-under-restriction transition constructor, analogous to the existing `one_bot` and `tensor_parr` synchronisation constructors.

The remaining synchronisation cases, such as `!w`, `!c`, `!?`, `$\exists\forall$` , `$\oplus_1\&$` , and `$\oplus_2\&$` , follow the same general restriction-synchronisation pattern as the multiplicative communication cases. They require isolating the interacting components, stepping the opened body, and reconstructing the resulting hyperenvironment up to permutation. Once the extraction lemmas for the current `res` case are

completed, these cases are expected to follow by analogous reasoning, with the relevant connectives and environments substituted for their multiplicative counterparts.

The duplication and disposal receive cases are more involved than the other server actions, since they generate a larger continuation process and manipulate dependencies and the typing environment. A prior revision of the formalisation, where server dependencies were stated implicitly, included the auxiliary constructions for building these processes and manipulating the corresponding dependencies and typing environments. Since the well-typedness of those constructions had already been established, parts of this earlier infrastructure may be reusable when extending the current dynamic metatheory.

7.3 Structural congruence

As discussed in [Section 6](#), the current formalisation does not include a process-level structural congruence relation. A future extension could define such a relation for the commutative, associative, and unit laws of parallel composition together with the required scope-manipulation principles for restriction, and then prove that it is compatible with typing and process transitions. This would address instances where well-typed processes become artificially stuck because dual actions are present, but their ordering, association, or placement under restriction does not match the form required by a synchronisation rule. An alternative design would be to incorporate selected symmetry, reassociation, or scope-management principles directly into the transition system rather than introducing an independent congruence relation.

Scope manipulation is especially delicate in the locally nameless setting, since moving a process under a restriction binder requires explicit de Bruijn index management. For example, a scope-extrusion principle of the form $(\nu((\bullet, \bullet)) P) \mid_p Q \equiv \nu((\bullet, \bullet)) (P \mid_p Q')$ would require Q' to be obtained by shifting the bound channel indices of Q by 2, while leaving its free names unchanged, to account for the two additional channel binders introduced by the restriction. This would require both a channel-level shift operation on bound indices and a process-level traversal lifting that operation through the syntax.

The trade-off is between modularity and local proof convenience. A full structural congruence relation would require preservation theorems showing that typing and transitions are stable under each congruence principle, but these results would then be reusable throughout the metatheory. A rule-oriented approach would instead add symmetric variants of selected rules, or selected symmetry and reassociation principles, directly to the transition relation. This could account for the structural flexibility most relevant to stepping while preserving the syntax-directed structure of the system, which is useful for proof search in Lean. However, these additional constructors would introduce extra cases in subsequent proofs. Although an unordered representation of parallel processes could avoid some commutativity issues, representing process syntax quotient-wise would make syntax-directed inversion and induction substantially harder in Lean, mirroring the considerations that led environments to be represented as lists rather than finite sets.

7.4 Readiness, productivity and behavioural metatheory

Beyond session fidelity, the operational metatheory of πLL includes results concerning the availability of interactions, the behaviour of infinite executions, and relations between concurrent process behaviours.

Readiness. One of the unmechanised metatheoretic objectives identified in the project description is progress. In the theory of [Montesi and Peressotti \[2021\]](#), this objective corresponds to the readiness theorem. Theorem 5.20 states that, whenever $\vdash P :: \Gamma_1 \mid \cdots \mid \Gamma_n$, every component Γ_i contains some name x such that P can perform an action whose free names include x . Thus,

readiness states that each component of the typed interface of a process offers at least one available interaction. Formalising this result would require proving that, for every component of a well-typed process's hyperenvironment, the process can currently perform a labelled transition involving a channel described by that component. The appropriate Lean proof architecture for establishing this correspondence remains to be developed.

Productivity and potential. The paper additionally proves productivity, which concerns the behaviour of infinite executions. Theorem 5.19 states that every infinite execution of a well-typed process contains infinitely many non-silent transitions. Equivalently, a well-typed process cannot execute internally forever while performing only finitely many observable actions. The proof uses a potential-annotated typing system: Lemma 5.18 bounds the length of any finite sequence of τ -transitions by the potential annotation, while Lemma 5.17 connects annotated typing derivations with ordinary typing derivations. Mechanising productivity would therefore require formalising the annotated typing system, its refinement result, and the potential argument used to bound internal computation. The design of this annotated layer and its integration with the present formalisation remain future work.

Further behavioural results. The paper also establishes behavioural and concurrency results that are not mechanised in the present development. These include congruence results for similarity and bisimilarity, the characterisation connecting similarity with typability, and the parallelism properties of the diamond property, serialisation, and non-interference. Similarity and bisimilarity provide relations for comparing process behaviour, while the parallelism properties describe how independent interactions may be reordered or shown not to interfere with one another. Mechanising these results would require introducing the corresponding behavioural relations and extending the transition-system development with the additional metatheoretic arguments from the paper. The organisation and proof-engineering requirements of such an extension remain to be investigated.

8 Conclusion

This thesis set out to mechanise the syntax, semantics, and metatheory of π LL in Lean 4. The aim was not to extend the source calculus, but to provide a mechanised account of an existing theoretical framework by making its treatment of binding, freshness, linear resources, and operational behaviour explicit within a proof assistant.

The formalisation covers the static structure of the full π LL calculus, including session types, processes, environments, hyperenvironments, and typing judgements. The binding architecture retains explicit free identifiers while representing bound occurrences by indices: channel binders are managed through locally nameless opening and co-finite premises, whereas polymorphic type binders are handled through type-depth indexing, shifting, and substitution. This representation avoids explicit reasoning up to α -equivalence for bound occurrences while preserving readable free channel names in processes, typing hyperenvironments, and transition labels.

The operational semantics is mechanised for the multiplicative fragment π MLL through three related transition systems: a primary transition relation over typing derivations, a projected process transition relation, and a projected environment transition relation describing the corresponding evolution of hyperenvironments. This organisation retains the derivation-level semantic architecture of the source theory while adapting its correspondence results to Lean's inductive-relational setting.

For the mechanised fragment, the development establishes the forward projection directions of the session-fidelity argument: typed derivation steps induce both process and environment transitions. The reverse lifting theorem, `typability_subject_reductionm`, is complete for the multiplicative transition cases except `res`. Consequently, the remaining obstacle to completing the

reverse direction of session fidelity for πMLL is concentrated in the restriction case, specifically in recovering the two endpoint-bearing hyperenvironment components required to reconstruct the enclosing cut derivation.

A central insight of the mechanisation is that choices which simplify the representation do not remove proof complexity, but relocate it. The locally nameless treatment of channel binders removes the need to reason explicitly about α -equivalent bound names, but introduces opening, local closure, and co-finite freshness infrastructure. Likewise, the paper presentation can treat fresh representatives and the rearrangement of unordered linear resources implicitly, whereas the Lean development must connect co-finite typing premises to the concrete names exposed by process transitions and must express hyperenvironment rearrangement through explicit permutation and extraction lemmas. The incomplete res case exposes this shift particularly clearly: the difficulty is not selecting fresh restricted endpoints or opening the body, but recovering the post-transition resource decomposition needed to reintroduce the restriction.

The development also identifies limitations and extensions beyond the current mechanisation. Environments and hyperenvironments are represented by lists modulo permutation, which supports structural induction but places a substantial burden on permutation-sensitive proofs. Process-level structural congruence is not yet formalised, so syntactic rearrangements available in the paper presentation are not available to the mechanised transition systems. Beyond completion of session fidelity for πMLL , extending the dynamic semantics to full πLL and mechanising readiness, productivity, and the broader behavioural metatheory remain future work.

Overall, the thesis provides a mechanised account of the static theory of full πLL and the operational metatheory of its multiplicative fragment. It confirms that the source theory’s derivation-level erasure and session-fidelity architecture can be represented in Lean, establishes the completed parts of that architecture, and isolates the structural infrastructure expected to close its remaining proof obligation.

References

- Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. Engineering Formal Metatheory. In *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Francisco, California, USA) (POPL ’08). ACM, New York, NY, USA, 3–15. doi:10.1145/1328438.1328443
- Brian E. Aydemir, Aaron Bohannon, Matthew Fairbairn, J. Nathan Foster, Benjamin C. Pierce, Peter Sewell, Dimitrios Vytiniotis, Geoffrey Washburn, Stephanie Weirich, and Steve Zdancewic. 2005. Mechanized Metatheory for the Masses: The PoplMark Challenge. In *Theorem Proving in Higher Order Logics*, Joe Hurd and Tom Melham (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 50–65.
- Luís Caires and Frank Pfenning. 2010. Session Types as Intuitionistic Linear Propositions. In *CONCUR*. 222–236.
- Marco Carbone, Sam Lindley, Fabrizio Montesi, Carsten Schürmann, and Philip Wadler. 2016. Coherence Generalises Duality: A Logical Explanation of Multiparty Session Types. In *27th International Conference on Concurrency Theory, CONCUR 2016, August 23–26, 2016, Québec City, Canada (LIPIcs, Vol. 59)*, Josée Desharnais and Radha Jagadeesan (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 33:1–33:15. doi:10.4230/LIPIcs.CONCUR.2016.33
- Arthur Charguéraud. 2012. The Locally Nameless Representation. *Journal of Automated Reasoning* 49, 3 (2012), 363–408. doi:10.1007/s10817-011-9225-2
- N. G. de Bruijn. 1972. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem. *Indagationes Mathematicae (Proceedings)* 75, 5 (1972), 381–392. doi:10.1016/1385-7258(72)90034-0
- Roberto Di Cosmo and Dale Miller. 2023. Linear Logic. In *The Stanford Encyclopedia of Philosophy* (fall 2023 ed.), Edward N. Zalta and Uri Nodelman (Eds.). Metaphysics Research Lab, Stanford University. <https://plato.stanford.edu/archives/fall2023/entries/logic-linear/>
- Jean-Yves Girard. 1987. Linear Logic. *Theor. Comput. Sci.* 50 (1987), 1–102. doi:10.1016/0304-3975(87)90045-4
- Robin Milner, Joachim Parrow, and David Walker. 1992. A Calculus of Mobile Processes, I. *Inf. Comput.* 100, 1 (1992), 1–40.
- Fabrizio Montesi and Marco Peressotti. 2021. Linear Logic, the π -calculus, and their Metatheory: A Recipe for Proofs as Processes. *CoRR* abs/2106.11818 (2021). arXiv:2106.11818 doi:10.48550/arXiv.2106.11818

Philip Wadler. 2014. Propositions as sessions. *Journal of Functional Programming* 24, 2–3 (2014), 384–418. Also: ICFP, pages 273–286, 2012.

A Code availability

The complete Lean development corresponding to this thesis is submitted as a separate archive and is available at: <https://github.com/MikkelBrixNielsen/LeanPiLL>

The submitted archive corresponds to the version of the formalisation described in this thesis. The online repository may contain later updates, while earlier experimental versions remain available in its history. These earlier versions are not part of the submitted artefact.

Since the complete executable development is provided in the accompanying archive, this appendix contains only selected definitions and proof fragments required to support the presentation in the thesis. These excerpts are typeset to closely reflect the corresponding Lean code, although some Unicode notation and spacing is rendered typographically rather than as directly executable source. The archived development contains the complete Lean files, including all definitions, auxiliary lemmas, and proof scripts.

B Lean file structure

The Lean development is organised into three main parts. The `Model/` directory contains the session types, processes, environments, hyperenvironments, and typing judgement. The `Semantics/` directory contains the transition labels and the process-, environment-, and judgement-level transition systems. The `SessionFidelity/` directory contains the main metatheoretic results relating these transition systems.

```
PiLL/
  Model/
    Environments/
    HyperEnvironments/
    Judgement/
    Processes/
    STypes/
    Base.lean
  Semantics/
    EnvStep/
    JudgementStep/
    ProcStep/
    Labels.lean
  SessionFidelity/
    ProcStep.lean
    EnvStep.lean
    Typability.lean
    SubjectReduction.lean
```

Listing 2. Top-level structure of the Lean development.

C Common Lean notation

The notation is purely an overloading layer. Each syntactic category supplies its own instance, while the displayed symbols allow the same operation to be written uniformly throughout the formalisation.

Type classes. The development uses a collection of typeclasses to overload recurring operations across syntactic categories. This allows shifting, substitution, opening, closing, prefix notation, and permutation to be written uniformly for types, processes, environments, hyperenvironments, actions, and labels. The following listing gives the common notation layer used throughout the later appendix excerpts.

```

class HasShift (Subject : Type) where
  shift : Subject → Nat → Nat → Subject

notation:max S:max " ↑ " d:max ", " c:max => HasShift.shift S d c
notation:max S:max " ↑ " c:max => HasShift.shift S 0 c
notation:max S:max "+" => HasShift.shift S 0 1

class HasSubst (Subject Replacement Target : Type) where
  subst : Subject → Replacement → Target → Subject

notation:max S "{ " R " // " T "}" => HasSubst.subst S R T

class HasOpen (Subject Replacement Target : Type) where
  open_ : Subject → Replacement → Target → Subject

notation:max S:max " (" T " | " R ")" => HasOpen.open_ S R T
notation:max S:max " (" R ")" => HasOpen.open_ S R 0

class HasClose (Subject Replacement Target : Type) where
  close_ : Subject → Replacement → Target → Subject

notation:max S:max " « " T " | " R " » " => HasClose.close_ S R T
notation:max S:max " « " R " » " => HasClose.close_ S R 0

class HasOpenTwo (Subject Replacement_1 Replacement_2 Target : Type) where
  open_ : Subject → Replacement_1 → Replacement_2 → Target → Subject

notation:max S:max " (" T " | " R1 " , " R2 ")" => HasOpenTwo.open_ S R1 R2 T
notation:max S:max " (" R1 " , " R2 ")" => HasOpenTwo.open_ S R1 R2 0

class HasCloseTwo (Subject Replacement_1 Replacement_2 Target : Type) where
  close_ : Subject → Replacement_1 → Replacement_2 → Target → Subject

notation:max S:max " « " T " | " R1 " , " R2 " » " => HasCloseTwo.close_ S R1 R2 T
notation:max S:max " « " R1 " , " R2 " » " => HasCloseTwo.close_ S R1 R2 0

class HasBracket (Subject Content Result : Type) where
  brack : Subject → Content → Result

class HasParen (Subject Content Result : Type) where
  paren : Subject → Content → Result

notation:80 x "[" "]" => HasBracket.brack x ( )
notation:80 x "[" "y" "]" => HasBracket.brack x y
notation:80 x "(" ")" => HasParen.paren x ( )
notation:80 x "(" (y) " " => HasParen.paren x y

```

```

class HasPerm ( $\alpha$  : Type) where
  perm :  $\alpha \rightarrow \alpha \rightarrow \mathbf{Prop}$ 

infixr:54 " ~ " => HasPerm.perm

```

Listing 3. Common typeclasses and notation used throughout the Lean development.

Type notation. The notation in the following listing presents the Lean constructors for session types using the symbols adopted throughout the thesis. In particular, it defines notation for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication, allowing later definitions and theorem statements to closely resemble their mathematical presentation.

```

instance : One Types := <Types.one>
instance : Bot Types := <Types.bot>
infixr:90 "  $\otimes$  " => Types.tensor
infixr:90 "  $\oplus$  " => Types.oplus
infixr:90 "  $\wp$  " => Types.parr
infixr:90 "  $\&$  " => Types.amp
prefix:95 "?? " => Types.quest
prefix:95 "!! " => Types.bang
prefix:max " $\exists$ ." => Types.exists_
prefix:max " $\forall$ ." => Types.forall_
postfix:max " $^\perp$ " => Types.dual
infix:90 "  $\multimap$  " => Types.linImpl

```

Listing 4. Notation for session-type constructors. These declarations provide concrete syntax for the linear-logic connectives, exponentials, quantifiers, duality, and linear implication.

Process notation. The following listing defines the concrete Lean notation used for process terms. The notation mirrors the process syntax introduced in the theoretical presentation, including prefix forms, restriction, branching, server actions, links, parallel composition, and the inactive process. This notation is used throughout the later typing and transition-system listings.

```

notation:80 x "[[]]." P => Proc.one x P
notation:80 x "[[•]]." P => Proc.tensor x P
notation:80 x "[[] A []]." P => Proc.output x P A
notation:80 x "[]()." P => Proc.bot x P
notation:80 x "[[•)]." P => Proc.parr x P
notation:80 x "[[◊)]." P => Proc.input x P

notation:75 " $\nu$ ([•,•])" P:80 => Proc.cut P
notation:80 x "[[L]]." P:80 => Proc.selectL x P
notation:80 x "[[R]]." P:80 => Proc.selectR x P
notation:80 x "[[USE]]." P:80 => Proc.consume x P
notation:80 x "[[DUP]]([•])." P:80 => Proc.duplicate x P
notation:80 x "[[DISP]]." P:80 => Proc.dispose x P
notation:80 "!" x ".{ " P:80 "}" => Proc.server x  $\emptyset$  P
notation:80 "!" x "< " zs ">.{ " P:80 "}" => Proc.server x zs P
notation:80 x ".case{L " " : " P:80 " , " "R" " : " Q :80 "}" => Proc.amp x P Q

```



```

notation:80 x "↔p" y => Proc.link x y
infixr:70 " |p " => Proc.par
notation "0" => Proc.nil

```

Listing 5. Notation for process constructors. These declarations provide concrete syntax for prefixes, restriction, branching, server actions, links, parallel composition, and the inactive process.

Label notation. The label syntax reuses the bracket and parenthesis notation introduced for process prefixes. Atomic actions instantiate the common `HasBracket` and `HasParen` interfaces, while generic instances lift this notation from `Act` to `Lbl`. Consequently, the same surface notation can denote either a process prefix or the corresponding transition label, depending on its expected type.

```

notation "L"      => Mu.L
notation "R"      => Mu.R
notation "USE"    => Mu.USE
notation "DUP"    => Mu.DUP
notation "DISP"   => Mu.DISP

@[simp] instance : HasBracket FPNName Unit Act where
  brack x _ := Act.one x
@[simp] instance : HasBracket FPNName FPNName Act where
  brack x y := Act.tensor x y
@[simp] instance : HasBracket FPNName Types Act where
  brack x A := Act.output x A
@[simp] instance : HasBracket FPNName Mu Act where
  brack x μ := Act.muBrack x μ

@[simp] instance : HasParen FPNName Unit Act where
  paren x _ := Act.bot x
@[simp] instance : HasParen FPNName FPNName Act where
  paren x y := Act.parr x y
@[simp] instance : HasParen FPNName Types Act where
  paren x A := Act.input x A
@[simp] instance : HasParen FPNName Mu Act where
  paren x μ := Act.muParen x μ

@[simp] instance {S C : Type} [HasBracket S C Act] :
  HasBracket S C Lbl where
    brack s c := Lbl.act (HasBracket.brack s c)

@[simp] instance {S C : Type} [HasParen S C Act] :
  HasParen S C Lbl where
    paren s c := Lbl.act (HasParen.paren s c)

notation:80 x " ↔1 " y => Lbl.link x y
notation:70 l " |1 " l' => Lbl.par l l'
notation:80 "τ" => Lbl.tau

```

Listing 6. Notation instances for actions and transition labels. Atomic action notation is lifted to labels so that transition labels mirror process-prefix notation.

D Selected core definitions

This appendix collects representative lemma statements for the core syntactic objects used throughout the formalisation. The listings are intended to show the shape of the mechanised syntax and judgements.

D.1 Session types

This listing gives the core Lean representation of session types. Type variables are represented using the locally nameless approach discussed in [Section 3.2](#), separating free type variables from bound de Bruijn indices.

```
abbrev Atom := Nat
abbrev FTVar := Nat
abbrev BTVar := Nat

inductive TVar : Type
| free    (x : FTVar)
| bound   (x : BTVar)
deriving DecidableEq, Repr

inductive Types : Type where
| atom      (a : Atom)
| atomDual  (a : Atom)
| var       (v : TVar)
| varDual   (v : TVar)
| one
| bot
| tensor    (A B : Types)
| parr      (A B : Types)
| oplus     (A B : Types)
| amp       (A B : Types)
| bang      (A : Types)
| quest     (A : Types)
| forall_   (A : Types)
| exists_   (A : Types)
deriving DecidableEq, BEq, Repr
```

Listing 7. Locally nameless representation of type variables and the Lean definition of session types.

D.2 Processes

This listing gives the core Lean representation of channels and process syntax. Channels are represented using the locally nameless approach discussed in [Section 3](#), separating free names from bound indices.

```
abbrev FPName := Nat
abbrev BPName := Nat

inductive Channel : Type where
| free    (x : FPName)
| bound   (x : BPName)
deriving DecidableEq, BEq, Repr
```

```

prefix: max "#" => Channel.free
prefix: max "$" => Channel.bound

inductive Proc : Type where
| nil
| one      (x : Channel) (P : Proc)
| bot      (x : Channel) (P : Proc)
| tensor   (x : Channel) (P : Proc)
| parr     (x : Channel) (P : Proc)
| cut      (P : Proc)
| par      (P Q : Proc)
| selectL  (x : Channel) (P : Proc)
| selectR  (x : Channel) (P : Proc)
| amp      (x : Channel) (P Q : Proc)
| output   (x : Channel) (P : Proc) (A : Types)
| input    (x : Channel) (P : Proc)
| server   (x : Channel) (zs : Finset Channel) (P : Proc)
| consume  (x : Channel) (P : Proc)
| duplicate (x : Channel) (P : Proc)
| dispose  (x : Channel) (P : Proc)
| link     (x y : Channel)
deriving DecidableEq, BEq

```

Listing 8. Locally nameless representation of channels and the Lean definition of process syntax.

D.3 Judgements

This listing gives the Lean definition of the main typing judgement, including the constructors for the static π LL typing relation and the exchange rules used to account for list-based environments.

```

inductive Typing : Nat → Proc → HyperEnv → Prop where
| mix0 {n : Nat} :
  Typing n 0 ∅

| mix {G H : HyperEnv} {P Q : Proc} {n : Nat} (hD : G.disjoint H) :
  Typing n P G → Typing n Q H →
  Typing n (P |p Q) (G |h H)

| one {P : Proc} {x : FName} {n : Nat} :
  Typing n P ∅ →
  Typing n (#x[]).P [[x:1]]

| bot {Γ : Env} {P : Proc} {x : FName} {n : Nat} (hF : x ∉ Γ.names) :
  Typing n P [Γ] →
  Typing n (#x()).P [x:⊥::Γ]

| cut {G : HyperEnv} {Γ Δ : Env} {P : Proc} {A : Types} {n : Nat}
(L : Finset FName) :
  (∀ x ∉ L, ∀ y ∉ L, x ≠ y →
    Typing n (P(#x, #y)) (G |h [x:A::Γ] |h [y:A⊥::Δ])) →
  Typing n (v(•,•) P) (G |h [Γ,Δ])

```

```

| tensor {Γ Δ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat}
  (hF : x ∉ Γ.names ∧ x ∉ Δ.names) (L : Finset FPName) :
  (∀ y ∉ L, Typing n (P((#y))) ([y:A::Γ] |h [x:B::Δ])) →
  Typing n (#x[•]).P [x:A⊗B::Γ,Δ]

| parr {Γ : Env} {P : Proc} {x : FPName} {A B : Types}
  {n : Nat} (hF : x ∉ Γ.names) (L : Finset FPName) :
  (∀ y ∉ L, Typing n (P((#y))) [y:A::x:B::Γ]) →
  Typing n (#x((•)).P) [x:A⋈B::Γ]

| oplus1 {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
  B.lc n → Typing n P [x:A::Γ] →
  Typing n (#x[L]).P [x:A⊕B::Γ]

| oplus2 {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
  A.lc n → Typing n P [x:B::Γ] →
  Typing n (#x[R]).P [x:A⊕B::Γ]

| amp {Γ : Env} {P Q : Proc} {x : FPName} {A B : Types} {n : Nat} :
  Typing n P [x:A::Γ] → Typing n Q [x:B::Γ] →
  Typing n (#x.case{L : P, R : Q}) [x:A&B::Γ]

| quest {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
  Typing n P [x:A::Γ] →
  Typing n (#x[USE]).P [x:??A::Γ]

| bang {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat} :
  ?eΓ → Typing n P [x:A::Γ] →
  Typing n (!#x(Γ.names.image Channel.free).(P)) [x:!!A::Γ]

| w {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
  (hF : x ∉ Γ.names) :
  A.lc n → Typing n P [Γ] →
  Typing n (#x[DISP]).P [x:??A::Γ]

| c {Γ : Env} {P : Proc} {x : FPName} {A : Types} {n : Nat}
  (hF : x ∉ Γ.names) (L : Finset FPName) :
  (∀ y ∉ L, Typing n (P((#y))) [x:??A::y:??A::Γ]) →
  Typing n (#x[DUP](•).P) [x:??A::Γ]

| exists_ {Γ : Env} {P : Proc} {x : FPName} {A B : Types} {n : Nat} :
  A.lc n → Typing n P [x:B{A // 0}::Γ] →
  Typing n (#x[A]).P [x:∃.B::Γ]

| forall_ {Γ : Env} {P : Proc} {x : FPName} {B : Types} {n : Nat} :
  Typing (n + 1) P [x:B::Γ+] →
  Typing n (#x(◊)).P [x:∀.B::Γ]

| ax {x y : FPName} {A : Types} {n : Nat} (hneq : x ≠ y) :
  A.lc n →
  Typing n (#x↔p #y) [x:A⊥:: [y:A]]

```

```

| exchange_env {G : HyperEnv} {Γ Δ : Env} {P : Proc} {n : Nat} :
  Typing n P (Γ::G) → Γ ~ Δ →
  Typing n P (Δ::G)

| exchange_hyper {G H : HyperEnv} {P : Proc} {n : Nat} :
  Typing n P G → G ~ H →
  Typing n P H

```

notation: $50 \ n \vdash P :: G \Rightarrow \text{Typing } n \ P \ G$

Listing 9. The main typing judgement. The judgement $\text{Typing } n \ P \ G$ states that process P is well typed under hyperenvironment G at type-variable depth n .

```

def proc {G : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P :: G) : Proc := P

def env {G : HyperEnv} {P : Proc} {n : Nat}
  (_ : n ⊢ P :: G) : HyperEnv := G

```

Listing 10. Projection functions from typing derivations. Since the judgement is indexed by both the process and hyperenvironment, these projections simply recover the corresponding indices.

D.4 Transition labels

This listing gives the core Lean definitions of the label and action datatypes discussed in the paragraph on labels in [Section 4.2](#), together with the associated free-name, introduced-name, and well-formedness functions used by the operational semantics.

```

inductive Mu : Type
| L
| R
| DISP
| DUP
| USE
deriving Repr, DecidableEq

inductive Act : Type
| one      (x : FPName)
| bot      (x : FPName)
| tensor   (x y : FPName)
| parr     (x y : FPName)
| output   (x : FPName) (A : Types)
| input    (x : FPName) (A : Types)
| muBrack  (x : FPName) (μ : Mu)
| muParen  (x : FPName) (μ : Mu)
deriving Repr, DecidableEq

inductive Lbl : Type
| tau
| link (x y : FPName)

```

```

| act (p : Act)
| par (l l' : Act)
deriving Repr, DecidableEq

@[simp] def fNamesAct : Act → Finset FPNName -- free names
| .one x | .bot x
| .tensor x _ | .parr x _
| .input x _ | .output x _
| .muBrack x _ | .muParen x _ => {x}

@[simp] def iNamesAct : Act → Finset FPNName -- introduced names
| .one _ | .bot _ | .input _ _ | .output _ _
| Act.muBrack _ _ | Act.muParen _ _ => ∅
| .tensor _ y | .parr _ y => {y}

@[simp] def Lbl.f : Lbl → Finset FPNName
| .tau => ∅
| .link x y => {x, y}
| .act a => fNamesAct a
| .par l l' => fNamesAct l ∪ fNamesAct l'

@[simp] def Lbl.i : Lbl → Finset FPNName
| .tau => ∅
| .link _ _ => ∅
| .act a => iNamesAct a
| .par l l' => iNamesAct l ∪ iNamesAct l'

@[simp] def Lbl.WF : Lbl → Prop
| .tau => True
| .link _ _ => True
| .act _ => True
| .par l l' => (iNamesAct l) ∩ (iNamesAct l') = ∅

```

Listing 11. Definitions of transition labels, actions, and their associated free and introduced names.

D.5 Transition systems

This listing collects the full inductive definitions of the three transition relations for π MLL, as referenced from [Section 4.2](#).

```

inductive TypingStepm :
  {n : Nat} → {G : HyperEnv} → {P : Proc} → Typing n P G → Lbl →
  {n' : Nat} → {G' : HyperEnv} → {P' : Proc} → Typing n' P' G' → Prop where
| one {P : Proc} {x : FPNName} {n : Nat} {D : Typing n P ∅} :
  TypingStepm (Typing.one (x := x) D) (x[]) D

| bot {Γ : Env} {P : Proc} {x : FPNName} {n : Nat}
  {hF : x ∉ Γ.names} {D : Typing n P [Γ]} :
  TypingStepm (Typing.bot (x := x) hF D) (x()) D

| tensor {Γ Δ : Env} {P : Proc} {x y : FPNName} {A B : Types} {n : Nat}
  {hF : x ∉ Γ.names ∧ y ∉ Δ.names} {L : Finset FPNName}

```

```

{huniq :  $\forall z, z \notin L \rightarrow \text{Typing } n \text{ (P(\#z))} ([z:A::\Gamma] \mid_h [x:B::\Delta])$ }
{hy :  $y \notin (\{x\} \cup P.f \cup \Gamma.\text{names} \cup \Delta.\text{names})$ } :
TypingStepm (Typing.tensor hF L huniq) (x[[y]]) (Typing_tensor_all_fresh huniq y hy
)

| parr { $\Gamma : \text{Env}$ } { $P : \text{Proc}$ } { $x \ y : \text{FPName}$ } { $A \ B : \text{Types}$ } { $n : \text{Nat}$ }
{hF :  $x \notin \Gamma.\text{names}$ } { $L : \text{Finset FPName}$ }
{huniq :  $\forall z, z \notin L \rightarrow \text{Typing } n \text{ (P(\#z))} [z : A::x : B::\Gamma]$ }
{hy :  $y \notin (\{x\} \cup P.f \cup \Gamma.\text{names})$ } :
TypingStepm (Typing.parr hF L huniq) (x((y))) (Typing_parr_all_fresh huniq y hy)

| par1 { $\mathcal{G} \ \mathcal{H} \ \mathcal{G}' : \text{HyperEnv}$ } { $P \ Q \ P' : \text{Proc}$ } { $l : \text{Lbl}$ } { $n : \text{Nat}$ }
{hd1 :  $\mathcal{G}.\text{disjoint } \mathcal{H}$ } {hd2 :  $\mathcal{G}'.\text{disjoint } \mathcal{H}$ }
{ $\mathcal{D} : \text{Typing } n \ P \ \mathcal{G}$ } { $\mathcal{D}' : \text{Typing } n \ P' \ \mathcal{G}'$ } { $\mathcal{E} : \text{Typing } n \ Q \ \mathcal{H}$ }
(h : TypingStepm  $\mathcal{D} \ l \ \mathcal{D}'$ ) (disj :  $l.i \cap Q.f = \emptyset$ ) :
TypingStepm (Typing.mix hd1  $\mathcal{D} \ \mathcal{E}$ )  $l$  (Typing.mix hd2  $\mathcal{D}' \ \mathcal{E}$ )

| par2 { $\mathcal{G} \ \mathcal{H} \ \mathcal{H}' : \text{HyperEnv}$ } { $P \ Q \ Q' : \text{Proc}$ } { $l : \text{Lbl}$ } { $n : \text{Nat}$ }
{hd1 :  $\mathcal{G}.\text{disjoint } \mathcal{H}$ } {hd2 :  $\mathcal{G}.\text{disjoint } \mathcal{H}'$ }
{ $\mathcal{D} : \text{Typing } n \ P \ \mathcal{G}$ } { $\mathcal{E} : \text{Typing } n \ Q \ \mathcal{H}$ } { $\mathcal{E}' : \text{Typing } n \ Q' \ \mathcal{H}'$ }
(h : TypingStepm  $\mathcal{E} \ l \ \mathcal{E}'$ ) (disj :  $l.i \cap P.f = \emptyset$ ) :
TypingStepm (Typing.mix hd1  $\mathcal{D} \ \mathcal{E}$ )  $l$  (Typing.mix hd2  $\mathcal{D} \ \mathcal{E}'$ )

| syn { $\mathcal{G} \ \mathcal{G}' \ \mathcal{H} \ \mathcal{H}' : \text{HyperEnv}$ } { $P \ P' \ Q \ Q' : \text{Proc}$ } { $l \ l' : \text{Act}$ } { $n : \text{Nat}$ }
{hd1 :  $\mathcal{G}.\text{disjoint } \mathcal{H}$ } {hd2 :  $\mathcal{G}'.\text{disjoint } \mathcal{H}'$ }
{ $\mathcal{D} : \text{Typing } n \ P \ \mathcal{G}$ } { $\mathcal{D}' : \text{Typing } n \ P' \ \mathcal{G}'$ }
{ $\mathcal{E} : \text{Typing } n \ Q \ \mathcal{H}$ } { $\mathcal{E}' : \text{Typing } n \ Q' \ \mathcal{H}'$ }
(h1 : TypingStepm  $\mathcal{D} \ l \ \mathcal{D}'$ ) (h2 : TypingStepm  $\mathcal{E} \ l' \ \mathcal{E}'$ )
(disj :  $(l \mid_{l'} l').i \cap (P \mid_{P'} Q).f = \emptyset$ ) (WF :  $(l \mid_{l'} l').\text{WF}$ ) :
TypingStepm (Typing.mix hd1  $\mathcal{D} \ \mathcal{E}$ )  $(l \mid_{l'} l')$  (Typing.mix hd2  $\mathcal{D}' \ \mathcal{E}'$ )

| one_bot { $\mathcal{G} : \text{HyperEnv}$ } { $\Gamma : \text{Env}$ } { $P \ P' : \text{Proc}$ } { $n : \text{Nat}$ } { $L : \text{Finset FPName}$ }
{huniq :  $\forall x \notin L, \forall y \notin L, x \neq y \rightarrow$ 
Typing  $n \text{ (P(\#x, \#y))} (\mathcal{G} \mid_h [x:1::\emptyset] \mid_h [y:\perp::\Gamma])$ }
{x y : FPName}
{hx :  $x \notin (\{y\} \cup P.f \cup \mathcal{G}.\text{names} \cup \Gamma.\text{names})$ }
{hy :  $y \notin (P.f \cup \mathcal{G}.\text{names} \cup \Gamma.\text{names})$ }
{ $\mathcal{D}' : \text{Typing } n \ P' \ (\mathcal{G} \mid_h [\emptyset, \Gamma])$ }
(hStep : TypingStepm (Typing_one_bot_all_fresh huniq x y hx hy)
(x[[[]]]  $\mid_{l_1} y((\ )) \ \mathcal{D}'$ ) :
TypingStepm (Typing.cut L huniq) ( $\tau$ )  $\mathcal{D}'$ 

| tensor_parr { $\mathcal{G} : \text{HyperEnv}$ } { $\Gamma \ \Delta \ \Xi : \text{Env}$ } { $P \ P' : \text{Proc}$ } { $A \ B : \text{Types}$ }
{ $n : \text{Nat}$ } { $L : \text{Finset FPName}$ }
{huniq :  $\forall x \notin L, \forall y \notin L, x \neq y \rightarrow$ 
Typing  $n \text{ (P(\#x, \#y))}$ 
 $(\mathcal{G} \mid_h [x:A \otimes B::\Gamma, \Delta] \mid_h [y:A^\perp \wp B^\perp::\Xi])$ }
{huniq' :  $\forall x \notin L, \forall y \notin L, x \neq y \rightarrow$ 
 $\forall x' \notin L, \forall y' \notin L, x' \neq y' \rightarrow$ 
 $x \neq x' \rightarrow x \neq y' \rightarrow y \neq x' \rightarrow y \neq y' \rightarrow$ 
Typing  $n \text{ (P'((2 \mid \#x, \#y))(\#x', \#y'))}$ 
 $(\mathcal{G} \mid_h [x:B::\Delta] \mid_h [x':A::\Gamma] \mid_h [y':A^\perp::y:B^\perp::\Xi])$ }

```

```

{ x y : FName } (hx : x ∉ L) (hy : y ∉ L) (hneq : x ≠ y)
{ x' y' : FName } (hx' : x' ∉ L) (hy' : y' ∉ L) (hneq' : x' ≠ y')
(hxx' : x ≠ x') (hxy' : x ≠ y') (hyx' : y ≠ x') (hyy' : y ≠ y')
(hxP : x ∉ P.f) (hyP : y ∉ P.f) (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
(hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f) (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
(hStep : TypingStepm (huniq x hx y hy hneq)
  (x[[x']] |1 y((y'))))
(huniq' x hx y hy hneq x' hx' y' hy' hneq' hxx' hxy' hyx' hyy')) :
TypingStepm (Typing.cut L huniq) (τ)
(Typing_double_cut_tensor_parr huniq')

| res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env} {P P' : Proc} {A : Types}
{ n : Nat } { l : Lbl } {L L' : Finset FName}
{ huniq : ∀ z ∉ L, ∀ w ∉ L, z ≠ w →
  Typing n (P((#z, #w))) (G |h [z:A::Γ] |h [w:A⊥::Δ]) }
{ huniq' : ∀ z ∉ L', ∀ w ∉ L', z ≠ w →
  Typing n (P'((#z, #w))) (G' |h [z:A::Γ'] |h [w:A⊥::Δ']) }
{ x y : FName } (hneq : x ≠ y)
(hx_pre : x ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
(hy_pre : y ∉ P.f ∪ G.names ∪ Γ.names ∪ Δ.names)
(hx_post : x ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
(hy_post : y ∉ P'.f ∪ G'.names ∪ Γ'.names ∪ Δ'.names)
(hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i)
(hStep : TypingStepm
  (Typing_res_all_fresh huniq x y hx_pre hy_pre hneq) l
  (Typing_res_all_fresh huniq' x y hx_post hy_post hneq)) :
TypingStepm (Typing.cut L huniq) l (Typing.cut L' huniq')

| perm_env {G H : HyperEnv} {Γ Γ' : Env} {P P' : Proc} {n n' : Nat} {l : Lbl}
{ D : n ⊢ P :: Γ :: G } {D' : n' ⊢ P' :: H} (hP1 : Γ ~ Γ')
(hTS : TypingStepm D l D') :
TypingStepm (Typing.exchange_env D hP1) l D'

| perm_hyper {G G' H H' : HyperEnv} {P P' : Proc} {n n' : Nat} {l : Lbl}
{ D : n ⊢ P :: G } {D' : n' ⊢ P' :: H}
(hP1 : G ~ G') (hP2 : H ~ H')
(hTS : TypingStepm D l D') :
TypingStepm (Typing.exchange_hyper D hP1) l
(Typing.exchange_hyper D' hP2)

```

Listing 12. The typed transition relation TypingStep_m.

```

inductive EnvStepm : HyperEnv → Lbl → HyperEnv → Prop where
| one {x : FName} :
  EnvStepm [[x:1]] (x[[ ]]) ∅

| bot {Γ : Env} {x : FName} :
  EnvStepm [x:⊥::Γ] (x(( ))) [Γ]

| tensor {Γ Δ : Env} {x x' : FName} {A B : Types}
  (hF : x' ∉ HyperEnv.names [x:A⊗B::Γ,Δ]) :
  EnvStepm [x:A⊗B::Γ,Δ] (x[[x']]) ([x':A::Γ] |h [x:B::Δ])

```



```

| parr {Γ : Env} {x x' : FPNName} {A B : Types}
  (hF : x' ∉ HyperEnv.names [x:A ⋈ B::Γ]) :
  EnvStepm [x:A ⋈ B::Γ] (x((x')) [x':A::x:B::Γ])

| par1 {G G' H : HyperEnv} {l : Lbl} :
  EnvStepm G l G' →
  EnvStepm (G |h H) l (G' |h H)

| par2 {G H H' : HyperEnv} {l : Lbl} :
  EnvStepm H l H' →
  EnvStepm (G |h H) l (G |h H')

| syn {G G' H H' : HyperEnv} {l l' : Act} :
  EnvStepm G l G' → EnvStepm H l' H' →
  EnvStepm (G |h H) (l |l l') (G' |h H')

| one_bot {G : HyperEnv} {Γ : Env} {x y : FPNName} :
  EnvStepm (G |h [[x:1]] |h [y:⊥::Γ]) (x[[ ]] |1y(( ))) (G |h [Γ]) →
  EnvStepm (G |h [Γ]) (τ) (G |h [Γ])

| tensor_parr {G : HyperEnv} {Γ Δ Ξ : Env}
  {x x' y y' : FPNName} {A B : Types} :
  EnvStepm
    (G |h [x:A ⊗ B::Γ,Δ] |h [y:A⊥ ⋈ B⊥::Ξ])
    (x[[x']] |1y((y')))
    (G |h [x:B::Δ] |h [x':A::Γ] |h [y':A⊥::y:B⊥::Ξ]) →
  EnvStepm (G |h [Γ,Δ,Ξ]) (τ) (G |h [Γ,Δ,Ξ])

| res {G G' : HyperEnv} {Γ Γ' Δ Δ' : Env} {x y : FPNName} {A : Types}
  {l : Lbl} (hFx : x ∉ l.i ∪ l.f) (hFy : y ∉ l.i ∪ l.f) :
  EnvStepm
    (G |h [x:A::Γ] |h [y:A⊥::Δ])
    l
    (G' |h [x:A::Γ'] |h [y:A⊥::Δ']) →
  EnvStepm (G |h [Γ,Δ]) l (G' |h [Γ',Δ'])

| perm_env {G H : HyperEnv} {Γ Γ' : Env} {l : Lbl} (hP : Γ ~ Γ') :
  EnvStepm (Γ::G) l H →
  EnvStepm (Γ'::G) l H

| perm_hyper {G G' H H' : HyperEnv} {l : Lbl} (hP1 : G ~ G') (hP2 : H ~ H') :
  EnvStepm G l H →
  EnvStepm G' l H'

```

Listing 13. The environment-level transition relation EnvStep_m .

inductive $\text{ProcStep}_m : (\text{P} : \text{Proc}) \rightarrow \text{Lbl} \rightarrow (\text{P}' : \text{Proc}) \rightarrow \text{Prop}$ **where**

```

| one {P : Proc} {x : FPNName} :
  ProcStepm (#x[[ ]].P) (x[[ ]]) P

```

```

| bot {P : Proc} {x : FPNName} :

```

```

ProcStepm (#x(()).P) (x(())) P

| tensor {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
  ProcStepm (#x[•].P) (x[y]) (P(#y))

| parr {P : Proc} {x y : FPNName} (hF : y ∉ {x} ∪ P.f) :
  ProcStepm (#x(•).P) (x(y)) (P(#y))

| par1 {P P' Q : Proc} {l : Lbl} :
  ProcStepm P l P' → l.i ∩ Q.f = ∅ →
  ProcStepm (P |p Q) l (P' |p Q)

| par2 {P Q Q' : Proc} {l : Lbl} :
  ProcStepm Q l Q' → l.i ∩ P.f = ∅ →
  ProcStepm (P |p Q) l (P |p Q')

| syn {P P' Q Q' : Proc} {l l' : Act} :
  ProcStepm P l P' → ProcStepm Q l' Q' →
  (l |l l').i ∩ (P |p Q).f = ∅ → (l |l l').WF →
  ProcStepm (P |p Q) (l |l l') (P' |p Q')

| one_bot {P P' : Proc} {x y : FPNName}
  (hx : x ∉ P.f) (hy : y ∉ P.f) (hneq : x ≠ y) :
  ProcStepm (P(#x, #y)) (x[[]] |l y()) P' →
  ProcStepm (v((•, •)) P) (τ) P'

| tensor_parr {P P' : Proc} {x x' y y' : FPNName}
  (hxP : x ∉ P.f) (hyP : y ∉ P.f)
  (hx'P : x' ∉ P.f) (hy'P : y' ∉ P.f)
  (hxP' : x ∉ P'.f) (hyP' : y ∉ P'.f)
  (hx'P' : x' ∉ P'.f) (hy'P' : y' ∉ P'.f)
  (hxx' : x ≠ x') (hxy : x ≠ y)
  (hxy' : x ≠ y') (hyx' : y ≠ x')
  (hyy' : y ≠ y') (hx'y' : x' ≠ y') :
  ProcStepm (P(#x, #y)) (x[x'] |l y(y')) (P'((2 | #x, #y))(#x', #y')) →
  ProcStepm (v((•, •)) P) (τ) (v((•, •)) (v((•, •)) P'))

| res {P P' : Proc} {l : Lbl} {x y : FPNName} (hneq : x ≠ y)
  (hx : x ∉ P.f ∪ P'.f) (hy : y ∉ P.f ∪ P'.f)
  (hlx : x ∉ l.f ∪ l.i) (hly : y ∉ l.f ∪ l.i) :
  ProcStepm (P(#x, #y)) l (P'(#x, #y)) →
  ProcStepm (v((•, •)) P) l (v((•, •)) P')

```

Listing 14. The process-level transition relation ProcStep_m .

E Session fidelity theorems

The following listings give the Lean statements of the completed erasure simulations and the target statements for typability subject reduction and subject reduction for π MLL, as discussed in [Section 5.1](#). The erasure functions `proc` and `env` ([Listing 10](#)) extract the process and hyperenvironment from a typing derivation, respectively.

The process- and environment-erasure simulations are complete. The proof of `typability_subject_reductionm` is complete for all multiplicative cases except `res`. Consequently, the final subject-reduction result remains conditional on completion of that case.

```
theorem session_fidelity_procm
  {n n' : Nat} {G G' : HyperEnv} {P P' : Proc} {l : Lbl}
  {D : Typing n P G} {D' : Typing n' P' G'} (hStep : TypingStepm D l D') :
  ProcStepm (proc D) l (proc D') := by ...
```

Listing 15. Process erasure simulation: typed derivation steps project to process steps.

```
theorem session_fidelity_envm
  {n n' : Nat} {G G' : HyperEnv} {P P' : Proc} {l : Lbl}
  {D : Typing n P G} {D' : Typing n' P' G'} (hStep : TypingStepm D l D') :
  EnvStepm (env D) l (env D') := by ...
```

Listing 16. Environment erasure simulation: typed derivation steps project to environment steps.

```
theorem typability_subject_reductionm
  {n : Nat} {G : HyperEnv} {P P' : Proc} {l : Lbl}
  (D : Typing n P G) (hPS : ProcStepm P l P') :
  ∃ (G' : HyperEnv) (D' : Typing n P' G'),
  TypingStepm D l D' := by ...
```

Listing 17. Target statement for typability subject reduction. The remaining incomplete proof case is `res`

```
theorem subject_reductionm {n : Nat} {G : HyperEnv} {P P' : Proc} {l : Lbl}
  (D : Typing n P G) (hPS : ProcStepm P l P') :
  ∃ G', EnvStepm G l G' ∧ (Typing n P' G') := by
  obtain ⟨G', D', hTS⟩ := typability_subject_reductionm D hPS
  have hES := session_fidelity_envm hTS
  exact ⟨G', hES, D'⟩
```

Listing 18. Target statement for subject reduction for the multiplicative fragment, conditional on completion of typability subject reduction.

F Selected proof infrastructure

This section collects representative lemma statements from the proof infrastructure supporting the session fidelity development, as discussed in [Section 5.1](#) and [Section 6](#).

Freshness bridge lemmas. The following lemmas bridge the mismatch between the co-finite quantification used in the typing rules and the concrete fresh names chosen by `ProcStepm`. They show that a co-finite typing premise can be instantiated at any concrete name that is fresh for the process and the surrounding environments. These lemmas are used by the typed transition rules for prefix actions, where the process transition records a particular name, while the typing rule provides a premise for all names outside a finite exclusion set.

```
lemma Typing_tensor_all_fresh {n : Nat} {P : Proc} {Γ Δ : Env}
  {x : FPNName} {A B : Types} {L : Finset FPNName}
  (hT : ∀ z ∉ L, Typing n (P(⟦z⟧)) ([z:A::Γ] |h [x:B::Δ])) :
```

```

  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names ∪ Δ.names) →
  Typing n (P(#y)) ([y:A::Γ] |h [x:B::Δ]) := by ...

```

```

lemma Typing_parr_all_fresh {n : Nat} {P : Proc} {Γ : Env}
  {x : FPNName} {A B : Types} {L : Finset FPNName}
  (hT : ∀ z ∉ L, Typing n (P(#z)) ([z:A::x:B::Γ])) :
  ∀ y, y ∉ ({x} ∪ P.f ∪ Γ.names) →
  Typing n (P(#y)) ([y:A::x:B::Γ]) := by ...

```

Listing 19. Freshness bridge lemmas for instantiating the co-finite premises of the tensor and parr typing rules.

All the *all-fresh* lemmas follow the same proof structure. They first choose the name or names required by the relevant constructor fresh from the co-finite exclusion set, the concrete target names, the free names of the process, and the names already present in the typing environment. The co-finite typing premise is then instantiated with these fresh names, and the resulting derivation is renamed to use the desired concrete names by applying substitution preservation for typing. The remaining proof obligations show that the substitutions have no effect on the surrounding typing environment or on the process body, except at the occurrences introduced by opening the process body. These obligations are discharged using the freshness assumptions together with the substitution and opening lemmas for processes and environments.

Typing inversion. Inversion lemmas recover the typing rule used to derive a given judgement. They are used throughout the typability proof to identify the typing rule compatible with the observed process shape.

The proofs proceed by induction on the typing derivation after first generalising the concrete process shape. All constructors whose conclusion cannot have the required process form are discharged by contradiction. The matching constructor case then uses injectivity of the process and channel constructors to recover the equalities between the displayed process and the process appearing in the typing derivation. For `Typing_inv_one`, this directly yields the empty continuation typing premise and a permutation showing that the hyperenvironment contains exactly the singleton environment for x . For `Typing_inv_tensor`, the matching case additionally exposes the hidden types, component environments, and co-finite premise of the original tensor rule. The exchange cases propagate the inversion result through environment and hyperenvironment permutation, composing the recovered permutation with the exchange permutation introduced by the typing derivation.

```

lemma Typing_inv_one {n : Nat} {P : Proc} {x : FPNName} {G : HyperEnv}
  (hT : Typing n (#x[]).P) G :
  (G ~ [[x:1]]) ∧ Typing n P ∅ := by ...

lemma Typing_inv_tensor {n : Nat} {P : Proc} {G : HyperEnv} {x : FPNName}
  (hT : Typing n (#x[•]).P) G :
  ∃ (A B : Types) (Γ Δ : Env) (L : Finset FPNName),
  (G ~ [x:A⊗B::Γ,Δ]) ∧ (∀ z ∉ L, Typing n (P(#z)) ([z:A::Γ] |h [x:B::Δ])) := by ...

```

Listing 20. Representative inversion lemmas for the typing relation. The one case recovers a singleton hyperenvironment and continuation typing derivation, while the tensor case recovers the hidden types, environments, and co-finite premise.

Hyperenvironment extraction. Extraction lemmas isolate specific components from a hyperenvironment permutation, exposing the typing environment required for a particular transition case. The following statement is representative of the extraction lemmas used when reconstructing a $\text{tensor_parr_synchronisation}$ beneath an enclosing restriction in the proof of $\text{typability_subject_reduction}_m$. Since hyperenvironments are represented as lists modulo permutation, the extracted components can appear in different relative positions. The lemma therefore returns a disjunction covering the possible arrangements.

The proof proceeds by extracting component membership from the pre-transition permutation, followed by case analysis on the relative positions of the restricted endpoints. Freshness and inequality assumptions rule out impossible coincidences, while the valid branches reconstruct the resulting hyperenvironment permutations using permutation rotation, merge, and cancellation lemmas.

Typed-transition name bounds. The following lemmas capture the name-level effect of typed transitions. The first states that a transition can introduce no names beyond those explicitly recorded as introduced by its label. The second states that names not involved in the action are preserved in the target hyperenvironment.

```

lemma TypingStepm_names_bound
  (hStep : TypingStepm  $\mathcal{D}$  1  $\mathcal{D}'$ ) :
   $\mathcal{G}'.$ names  $\subseteq \mathcal{G}.$ names  $\cup$  1.i := by ...

lemma TypingStepm_names_preserved
  (hStep : TypingStepm  $\mathcal{D}$  1  $\mathcal{D}'$ ) :
   $\mathcal{G}.$ names  $\setminus$  (1.f  $\cup$  1.i)  $\subseteq \mathcal{G}'.$ names := by ...

```

Listing 21. Name-evolution bounds for typed transitions. A transition introduces no names beyond those recorded by its label, and preserves all source names not involved in the action.

Both proofs proceed by induction on the typed transition derivation. The prefix axioms reduce to finite-set calculations on the names consumed or introduced by their respective labels. The parallel constructors lift the induction hypothesis through hyperenvironment merge, using monotonicity of union and subset. The permutation cases use the fact that permutation preserves the name set of a hyperenvironment. In the restriction case, the induction hypothesis is applied to the transition of the opened body, giving a name bound in terms of the opened hyperenvironments. The freshness assumptions on the restricted endpoints, stating that neither appears in the label, then allow the temporary names introduced by opening to be cancelled from both sides, recovering the bound on the outer closed hyperenvironments.

Block preservation lemmas. The following lemmas establish that environment components not involved in a transition persist in the post-state hyperenvironment up to permutation. The fully proved lemma $\text{TypingStep}_m\text{-preserves_block}$ states that any component whose names are disjoint from the label survives the transition. Building on this, $\text{TypingStep}_m\text{-preserves_single_block}$ tracks a specific component carrying a distinguished endpoint through a transition, with its remaining proof obligations confined to its res case. The lemma $\text{TypingStep}_m\text{-preserves_disjoint_or_merges}$ simultaneously tracks two dual endpoint components, concluding either that they remain in separate post-state components or that they have been merged into one. This disjunction captures the two structural configurations that may result from stepping beneath a restriction, and is intended to provide the post-transition hyperenvironment structure needed to complete the res case of $\text{typability_subject_reduction}_m$ by rebuilding the outer cut derivation. The proofs proceed

by induction on the typed transition derivation, transporting unaffected components directly or through permutation and applying the induction hypothesis to components in an evolving branch. In the nested restriction argument, the stronger lemmas additionally rely on the completed block-preservation result to recover endpoint-bearing components beneath the inner transition.

```

lemma TypingStepm_preserves_block
  {n n' : Nat} {P P' : Proc} {G G' : HyperEnv} {l : Lbl} {Γ : Env}
  {D : n ⊢ P :: G} {D' : n' ⊢ P' :: G'}
  (hStep : TypingStepm D l D') (hΓ : Γ ∈ G)
  (hD : Disjoint Γ.names (l.f ∪ l.i)) :
  ∃ Γ' ∈ G', Γ' ~ Γ := by ...

lemma TypingStepm_preserves_single_block
  {n n' : Nat} {P P' : Proc} {G G' H : HyperEnv} {Γ : Env}
  {A : Types} {x y : FPName} {l : Lbl}
  {D' : n' ⊢ P' :: H} (D : n ⊢ P :: G) (hStep : TypingStepm D l D')
  (hx1 : x ∉ l.f ∪ l.i) (hP : G ~ G' |h [x:A::Γ]) :
  ∃ G'' Γ', H ~ G'' |h [x:A::Γ'] := by ...

lemma TypingStepm_preserves_disjoint_or_merges
  {n n' : Nat} {P P' : Proc} {G G_pre H : HyperEnv} {Γ Δ : Env}
  {A : Types} {x y : FPName} {l : Lbl}
  {D' : n' ⊢ P' :: H} (D : n ⊢ P :: G_pre) (hStep : TypingStepm D l D')
  (hx1 : x ∉ l.f ∪ l.i) (hy1 : y ∉ l.f ∪ l.i)
  (hP : G_pre ~ G |h [x:A::Γ] |h [y:A⊥::Δ]) :
  (∃ G' Γ' Δ', H ~ G' |h [x:A::Γ'] |h [y:A⊥::Δ']) ∨
  (∃ G' Δ', H ~ G' |h [x:A::Δ'], [y:A⊥]) := by

```

Listing 22. Block preservation lemmas for typed transitions.

```

lemma HyperEnv.Perm.extract_tensor_parr_res
  {G H Gr : HyperEnv} {Γ Γ' Γ'' Δ Δ' Δ'' Δ' : Env}
  {u v x x' y y' : FPName} {A B C D E : Types}
  (h_pre : G |h [u:E::Γ'] |h [v:E⊥::Δ'] ~
    Gr |h [x:A ⊗ B::Γ, Δ] |h [y:C ⊗ D⊥::Δ'])
  (h_post : H |h [u:E::Γ''] |h [v:E⊥::Δ''] ~
    Gr |h [x:B::Δ] |h [x':A::Γ] |h [y':C::y:D::Δ])
  (hux : u ≠ x) (hux' : u ≠ x') (huy : u ≠ y) (huy' : u ≠ y')
  (hvx : v ≠ x) (hvx' : v ≠ x') (hvy : v ≠ y) (hvy' : v ≠ y')
  (huv : u ≠ v) (hFu : u ∉ G.names) (hFv : v ∉ G.names)
  (hFu' : u ∉ H.names) (hFv' : v ∉ H.names)
  (huΔ : u ∉ Δ.names) (hvΓ' : v ∉ Γ'.names) :
  (∃ Gn Γn Δn Δn,
    G |h [Γ', Δ'] ~ Gn |h [x:A ⊗ B::Γn, Δn] |h [y:C ⊗ D⊥::Δn] ∧
    H |h [Γ'', Δ''] ~ Gn |h [x:B::Δn] |h [x':A::Γn] |h [y':C::y:D::Δn])
  ∨
  (∃ Gn Γn Δn Δn,
    G |h [Γ', Δ'] ~ Gn |h [x:A ⊗ B::Γn, Δn ++ y:C ⊗ D⊥::Δn] ∧
    (H |h [Γ'', Δ''] ~ Gn |h [x:B::Δn] |h [x':A::Γn ++ y':C::y:D::Δn] ∨
    H |h [Γ'', Δ''] ~ Gn |h [x':A::Γn] |h [x:B::Δn ++ y':C::y:D::Δn])) := by ...

```

Listing 23. Representative hyperenvironment extraction lemma for the tensor-parr synchronisation case.